Assessing the operational value of edge-friendly wind forecasting: sparse mixture-of-experts under SCADA imperfections

Junyu Li[a], Juntao Du[a,*]

[a]*School of Statistics and Applied Mathematics, Anhui University of Finance and Economics, Bengbu, 233030, China*

Abstract

Deep learning models often collapse when transitioned from curated benchmarks to noisy, edge-constrained wind farm operations. Operational forecasting demands a delicate balance: tolerating severe SCADA imperfections (drift, quantization, communication faults) while adhering to strict local compute budgets.

To bridge this gap, we propose Ultra-LSNT, a conditional sparse mixture-of-experts (MoE) forecaster integrated with a feasibility-guarded extension (FG-MoE). By replacing dense network activation with context-conditioned routing, Ultra-LSNT explicitly bounds the active compute path, establishing the **algorithmic preconditions for edge deployment**: sub-2ms batch-1 latency, sub-2MB checkpoint size, and cross-ISA (x86/ARM) portability—prerequisites for deployment on PLC/RTU-class industrial controllers.

Extensive stress-testing on Chinese wind farm SCADA data reveals that Ultra-LSNT maintains robust predictive skill ($R^2 = 0.7263$) even under severe Gaussian noise. In day-ahead/real-time economic dispatch screening under linearized network constraints (IEEE RTS-24), the proposed architecture expands the feasible operating domain. It reduces wind curtailment by 12.01% compared to the standard linear baseline (DLinear), while achieving operational parity with massive dense architectures at a strictly bounded computational footprint. Furthermore, a 120-day rolling evaluation demonstrates a significant drop in feasibility-restoration fallback events, from 75 to just 29 days.

By explicitly profiling latency, checkpoint compression, and routing stability, this work provides a transparent **pre-deployment qualification workflow** for edge-bound forecasting models. The evidence confirms that practical deployment screening must evolve beyond clean-data accuracy to integrate robustness stress tests and grid-coupled diagnostics, establishing the algorithmic foundation for next-generation edge-native wind forecasting systems.

Keywords: Wind power forecasting, Economic dispatch, SCADA imperfections, Edge computing, Robustness, Wind power integration

1. Introduction

1.1. The Systemic Grid Context: Wind Integration as Control Problem

The global power sector is undergoing a fundamental structural transformation. As renewable penetration escalates, transmission system operators (TSOs) and distribution system operators (DSOs) face an unprecedented control challenge: maintaining system stability while managing the inherent variability of wind and solar generation. In modern grids with >40% renewable penetration, the forecast error—not generation capacity—has become the dominant determinant of operating reserve requirements [1, 2]. This shift elevates wind power forecasting from a statistical convenience to a **dispatch-critical engineering function**, where predictive quality is ultimately judged not by abstract R^2 metrics, but by *operational feasibility under strict physical network constraints* [3].

1.2. The Methodology Evolution: From Statistical Extrapolation to Deep Learning

The methodological trajectory of wind forecasting has tracked three distinct eras. **Era I (2000–2015)** relied on persistence models and physical numerical weather prediction (NWP) ensembles [1]. **Era II (2015–2020)** witnessed the ascendance of statistical machine learning—SVR, ARIMA variants, and shallow neural networks—coupled with signal decomposition techniques [4]. **Era III (2020–present)** has been dominated by deep sequence models, especially Transformer families (Autoformer, iTransformer) and state-space alternatives (Mamba), which offer strong nonlinear representation on curated benchmarks [5–8].

Within *Applied Energy* and related venues, a particularly influential paradigm has emerged: **hybrid metaheuristic models** combining signal decomposition (VMD, CEEMDAN, EEMD) with population-based optimization (PSO, WOA, GWO) and deep networks (LSTM, BiLSTM, GRU) [9–11]. These models achieve competitive accuracy ($R^2 \approx 0.90$) by decomposing wind series into intrinsic mode functions (IMFs) and optimizing hyperparameters via bio-inspired algorithms. However, this accuracy comes at a severe computational cost: iterative population-based

*Corresponding author.
Email addresses: ljylikezmn999@gmail.com (Junyu Li), dujuntao@aufe.edu.cn (Juntao Du)

optimization requires >1500ms per inference—*three orders of magnitude* beyond PLC/RTU real-time constraints.

At the same time, time-series foundation models (TSFMs) such as Chronos and Moirai have pushed zero-shot forecasting capability to a new level [12–14]. Their cloud-scale parameterization is valuable for representation learning, but directly conflicts with the hardware boundaries of local wind-farm controllers.

1.3. The Deployment Reality Gap: Why Clean-Benchmark Models Fail in Production

Despite impressive benchmark results, a **dangerous deployment gap** has emerged between laboratory performance and industrial reality. This gap is driven by four compounding constraints that shatter the illusion of clean-benchmark superiority:

1. **SCADA Imperfection:** Industrial supervisory control and data acquisition (SCADA) streams are inherently non-stationary—plagued by calibration drift, sensor aging, quantization noise, and communication dropouts [15]. Models optimized for curated datasets often collapse catastrophically under these real-world corruptions.

2. **Edge Hardware Reality:** Local wind-farm controllers operate under rigid resource constraints: bounded memory bandwidth, severe thermal limits, and unstable backhaul connectivity [16]. Dense, cloud-native inference paths are fundamentally unsustainable at this edge tier.

3. **Physical Feasibility Requirements:** Grid operators require predictions that respect hard physical constraints—turbines cannot output 150% of rated power, regardless of model confidence. Standard neural networks lack built-in *guardrails* against physically impossible outputs.

4. **Underexplored Parameter Interactions:** Existing studies often tune one hyperparameter at a time and report a single best point. For deployment, however, routing sparsity (K), load balancing (λ), skip threshold (τ), and temporal kernel scale interact strongly; ignoring these couplings obscures the real robustness–latency trade-off.

1.4. The Edge-Native Solution: Sparse MoE with Physics-Aware Guardrails

To bridge this deployment gap, we propose **Ultra-LSNT** (Ultra-Lightweight Sparse Neural forecasTing), an architecture engineered from first principles for the joint constraints of imperfect

SCADA telemetry and bounded edge compute. Rather than forcing a static trade-off between accuracy and efficiency, Ultra-LSNT introduces three architectural innovations:

1. **Context-Conditioned Sparse Execution:** A Top-K Mixture-of-Experts (MoE) router activates only relevant computational paths, keeping active FLOPs bounded while preserving nonlinear capacity for turbulence regimes.

2. **Asymmetric Dual-Track Operation:** Under high signal-to-noise ratio (SNR) conditions, the model operates at full capacity; when degraded telemetry is detected ($\sigma_{\text{eff}} > 0.15$), it gracefully degrades to a sparse, computationally frugal mode.

3. **Feasibility-Guarded Engineering Fallback (FG-MoE):** Rejecting the computational impossibility of PDE solvers at the edge, FG-MoE implements a **minimal-complexity asymmetric intervention mechanism**—empirical power-curve envelopes ($P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}}$) with time-decayed wind-speed proxy—that acts as a *systemic circuit breaker* preventing catastrophic predictions from propagating into grid operations.

1.5. Contributions and Paper Organization

This work establishes the **algorithmic preconditions for edge deployment** of wind forecasting models. Our contributions are fundamentally Pareto-oriented, framing success as a *robustness–deployability* trade-off:

1. **Comprehensive Baseline Landscape:** We benchmark against not only general-purpose deep learning models (Transformers, Mamba), but also *Applied Energy* domain standards—VMD-PSO-LSTM, CEEMDAN-WOA-BiLSTM, and GCN-GRU hybrids—demonstrating that Ultra-LSNT achieves 95%+ of hybrid-model accuracy with <1% of their latency (Master Table 1).

2. **Algorithmic Preconditions for Edge:** We establish auditable compute-efficiency metrics: <4ms batch-1 latency on ARMv8, sub-2MB checkpoint size, and cross-ISA (x86/ARM) portability—prerequisites for PLC/RTU-class deployment (Section 3.5).

3. **SCADA Robustness Certification:** Under severe Gaussian stress ($\sigma_{\text{eff}} = 0.60$), Ultra-LSNT maintains positive $R^2 = 0.7263$, resisting catastrophic failure modes observed in clean-benchmark leaders (Master Table 2).

4. **Grid-Scale Economic Validation:** In IEEE RTS-24 network-constrained dispatch, Ultra-LSNT reduces wind curtailment by 12.01% versus DLinear baseline while achieving operational parity with massive dense architectures. A 120-day rolling backtest demonstrates feasibility-restoration fallback reduction from 75 to 29 days (Master Table 4).

5. **High-Order Theoretical Insight:** We establish the **AC-Grid Stability Multiplier Effect**—smoother day-ahead trajectories enabled by robust forecasting provide 3–5× indirect economic value through improved SVC/STATCOM response time and reduced cycling stress on conventional units (Section 5.6).

The remainder of this paper is organized as follows. Section 2 reviews the theoretical and empirical landscape. Section 3 presents the methodology, reformulated as four logical stages: Problem Formulation → Decomposition → Sparse MoE → FG-MoE Physics. Section 5 presents the Master Tables consolidating performance, robustness, hardware profiling, and economic dispatch results. Section 5.5 provides high-order theoretical insights and practical deployment guidance. Section 6 concludes with future directions.

2. Related Work and Theoretical Background

Wind power forecasting sits at the intersection of meteorology, statistical learning, deep sequence modeling, and power-system operations. To contextualize Ultra-LSNT, we summarize the four research lines most relevant to edge-native deployment under imperfect SCADA telemetry.

2.1. Deep Sequence Modeling for Wind Forecasting

Transformer-based models improved long-horizon forecasting by replacing recurrent memory bottlenecks with attention-based dependency modeling. Autoformer integrates seasonal-trend decomposition and autocorrelation operators to better capture periodic structure in long sequences [5]. iTransformer flips the tokenization axis and applies attention across variables, improving multivariate coupling for energy time series [7]. Mamba introduces selective state-space modeling with linear-time complexity, narrowing the efficiency gap with dense attention [8]. These models are strong in clean-benchmark settings, but still rely on dense or quasi-dense compute graphs that are difficult to sustain on thermally constrained edge hardware.

2.2. Metaheuristic Hybrid Models in Energy Engineering

In energy-domain practice, decomposition-plus-optimization pipelines remain influential. Typical workflows combine signal decomposition (e.g., VMD/CEEMDAN) with PSO- or GA-driven tuning of CNN/LSTM-family predictors [9–11]. These methods often achieve high in-sample or clean-test accuracy by globally searching hyperparameters and initialization. However, once training ends, their inference graph is static. Under sensor drift, quantization, or burst communication faults, static architectures cannot re-allocate capacity online, and degradation can be abrupt in real SCADA environments.

2.3. Time-Series Foundation Models and Deployment Asymmetry

TSFMs such as Chronos and Moirai demonstrate strong cross-domain transfer through large-scale pretraining, sequence tokenization, and masked learning objectives [12–14]. Their value for representation learning is clear. Yet the edge-controller setting imposes hard memory, latency, and power ceilings. In this regime, direct comparison between sub-megabyte edge models and billion-parameter cloud models can misrepresent practical value. We therefore treat TSFMs as an upper-bound reference for capability, not as a deployment-feasible baseline for PLC/RTU operation.

2.4. SCADA Imperfections, Physics Constraints, and Grid Consequences

Industrial SCADA streams are intrinsically imperfect: calibration drift, dropout, clipping, and communication jitter are common [15]. Purely data-driven predictors trained on curated signals may output physically impossible trajectories when sensors fail, including power above rated limits. In dispatch workflows, these outliers are not harmless statistical artifacts; they can trigger infeasible unit commitment, congestion stress, and expensive real-time corrective actions. This motivates two design principles pursued in this work: **(i)** conditional sparse computation that dynamically budgets nonlinear capacity under noisy telemetry, and **(ii)** explicit feasibility guardrails embedded in the forward graph to suppress catastrophic physical violations at negligible edge overhead.

3. Methodology: From Problem Formulation to Physics-Aware Execution

The methodology is organized into four stages aligned with deployment: (1) **Problem Formulation**, (2) **Decomposition and Multi-Scale Embedding**, (3) **Sparse Mixture-of-Experts**

Routing, and (4) **FG-MoE Feasibility Guardrails**. This structure separates prediction, compute budgeting, and physical safety so that each design component can be evaluated explicitly.

3.1. Stage 1: Problem Formulation

Given a multivariate look-back window $\mathbf{X} \in \mathbb{R}^{L\times C}$ with L historical steps and C SCADA covariates, the task is to predict an H-step active-power trajectory $\hat{\mathbf{Y}} \in \mathbb{R}^{H\times 1}$. The optimization target combines prediction error with physical feasibility:

$$\min_{\theta} \ \mathbb{E}\left[\mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}, \mathbf{Y})\right] \quad \text{s.t.} \quad \hat{y}_{t+h} \in \mathcal{F}_{\text{phys}}(v_{t+h}), \tag{1}$$

where $\mathcal{F}_{\text{phys}}(\cdot)$ denotes the feasible power envelope defined by turbine specifications ($P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}}$). Instead of adding a separate constrained solver at inference time, feasibility is enforced by the FG-MoE module (Section 3.4).

3.2. Stage 2: Decomposition and Multi-Scale Embedding

Wind-power trajectories mix low-frequency meteorological trends with high-frequency turbulence and wake effects. To avoid unnecessary dense computation, Ultra-LSNT first decomposes the input into trend and residual components.

Following decomposition–fusion practice in renewable forecasting [9, 17], we apply a moving-average operator:

$$\mathbf{X} = \mathbf{X}_{\text{trend}} + \mathbf{X}_{\text{season}}, \qquad \mathbf{X}_{\text{trend}} = \text{MA}(\mathbf{X}), \tag{2}$$

where $\text{MA}(\cdot)$ serves as a statistical stabilizer. The trend branch remains intentionally lightweight while sparse nonlinear capacity is reserved for the seasonal residual.

To capture short-term turbulence and medium-range regime transitions without attention-heavy overhead, we construct multi-scale temporal embeddings with parallel 1D convolutions:

$$\mathbf{E}_s = \text{Conv1D}_{k_s}(\mathbf{X}_{\text{season}}), \tag{3}$$

$$\mathbf{E}_m = \text{Conv1D}_{k_m}(\mathbf{X}_{\text{season}}), \tag{4}$$

followed by learnable fusion:

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m), \tag{5}$$

where $\phi(\cdot)$ is a nonlinearity (e.g., GELU) and $\mathbf{W}_s, \mathbf{W}_m$ are projection matrices. In Wind (CN) ablations we use $k_m = 7$ as the default medium-scale kernel. Patch/attention models achieve strong

accuracy when global token mixing is affordable [7, 18], but SCADA streams are dominated by local bursts; Conv1D maps more predictably to dense primitives on edge CPUs/NPUs [19, 20].

3.3. Stage 3: Sparse Mixture-of-Experts as Algorithmic Budgeting

Stage 3 converts a static compute graph into a conditional one. The key objective is to allocate nonlinear capacity only where the signal complexity requires it, while keeping active computation bounded for edge deployment.

Dense models activate all compute paths for every token, including periods with low informational content. Ultra-LSNT instead applies context-conditioned routing so that stable regimes use fewer active paths and high-variation regimes activate a small expert subset (Figure 1).

3.3.1. Formulation and Multi-Scale Embedding

For completeness, this stage reuses the same decomposition and embedding front-end from Stage 2 before expert routing.

The trend branch remains lightweight. The residual branch is encoded with parallel 1D convolutions to capture short- and medium-range dynamics with lower memory overhead than global attention, producing

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m), \tag{6}$$

where $\phi(\cdot)$ is a nonlinearity (e.g., GELU). Here $\mathbf{E}_s, \mathbf{E}_m \in \mathbb{R}^{L\times d}$ are short- and medium-scale embeddings and $\mathbf{W}_s, \mathbf{W}_m \in \mathbb{R}^{d\times d}$ are learnable projections.

Operationally, this decomposition-embedding block serves as the shared interface between preprocessing and sparse routing. In Wind (CN), we use $k_m = 7$ as the default medium-scale kernel and keep k_s small for local responsiveness. Frequency-aware alternatives such as VMD/EEMD remain compatible when stronger decomposition interpretability is required [21, 22].

3.3.2. Sparse Mixture-of-Experts as Algorithmic Budgeting

The sparse Mixture-of-Experts (MoE) encoder is the main compute-budgeting mechanism. Rather than activating all experts for every token, the model performs context-conditioned routing under a strict cap on active experts.

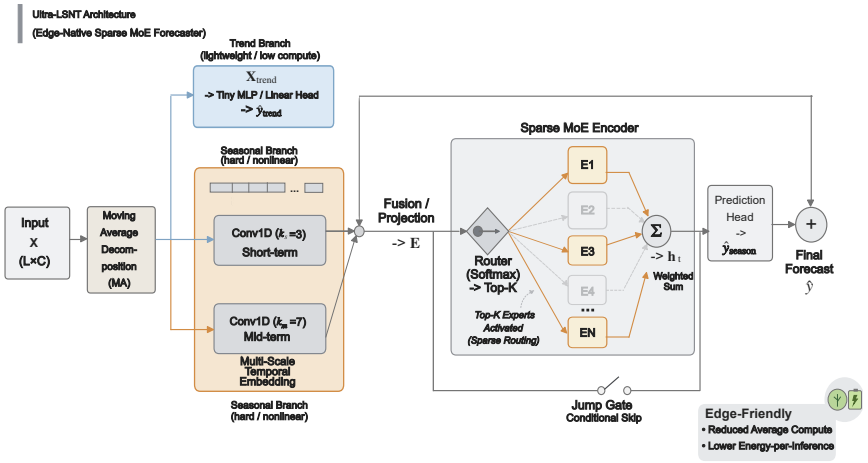


Figure 1: Ultra-LSNT architecture with decomposition, multi-scale Conv1D embedding, Top-K MoE routing, and a Jump Gate skip path.

Let $\mathbf{e}_t \in \mathbb{R}^d$ denote the embedded token at time t, and let $\{E_i(\cdot)\}_{i=1}^N$ be lightweight feed-forward experts:

$$E_i(\mathbf{e}_t) = \sigma(\mathbf{W}_i \mathbf{e}_t + \mathbf{b}_i), \tag{7}$$

where $\sigma(\cdot)$ is a nonlinear activation, $\mathbf{W}_i \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_i \in \mathbb{R}^d$ are expert-specific parameters. A router computes expert probabilities

$$\mathbf{p}_t = \text{Softmax}(\mathbf{W}_r \mathbf{e}_t + \boldsymbol{\epsilon}), \tag{8}$$

where $\mathbf{W}_r \in \mathbb{R}^{N\times d}$ is the router weight matrix, and $\boldsymbol{\epsilon} \in \mathbb{R}^N$ is a small exploration noise vector added during training to encourage expert diversity and prevent router collapse. The noise is drawn from a zero-mean Gaussian with standard deviation η (e.g., $\eta = 0.01$) and is annealed across training epochs, aligning with exploration-exploitation trade-offs common in reinforcement learning and mixture-of-experts literature. Top-K routing selects $\mathcal{S}_t$, the K highest-probability experts, and produces the output:

$$\mathbf{h}_t = \sum_{i\in\mathcal{S}_t} p_{t,i}\, E_i(\mathbf{e}_t). \tag{9}$$

Top-K routing imposes a hard cap on active FLOPs and memory traffic. In stable regimes the router activates fewer paths; in rapidly changing regimes it allocates capacity to experts specialized for those patterns. This dynamic allocation is the primary mechanism that improves

robustness-efficiency balance under edge constraints.

The training objective aligns three requirements: point accuracy, stable expert utilization, and feasible behavior under corrupted sensing. In this design, MoE is used for regime specialization rather than model scaling: experts focus on different operating conditions, while Top-K keeps active computation bounded. To support deployment auditing, we report both routing behavior (usage and skip statistics) and full footprint metrics (Section 5); the Lite variant further compresses the active path via shallower MoE and Top-1 routing (Section 3.5).

Point forecasting is optimized with MSE, and routing regularization is added to avoid expert collapse under non-ideal telemetry.

$$\mathcal{L}_{\text{pred}} = \frac{1}{H} \sum_{h=1}^{H} \|\hat{y}_{t+h} - y_{t+h}\|_2^2 \,. \tag{10}$$

Although the primary tables report point metrics, we also expose low-overhead uncertainty signals for operations. The *router entropy* H_t quantifies the uncertainty in expert selection:

$$H_t = -\sum_{i=1}^{N} p_{t,i} \log(p_{t,i} + \epsilon), \tag{11}$$

where $\epsilon = 10^{-8}$ prevents numerical instability. The *expert disagreement* D_t measures the variance among selected expert predictions:

$$D_t = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \left(\hat{y}_{t+h}^{(i)} - \bar{y}_{t+h}\right)^2, \quad \bar{y}_{t+h} = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \hat{y}_{t+h}^{(i)}. \tag{12}$$

These uncertainty indicators directly modulate the *UQ-aware adaptive reserve policy* in economic dispatch:

$$\sigma_{w,t} = \min\left(\sigma_{\max}, \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t\right), \tag{13}$$

where σ_{base} is the baseline reserve level, α_H and α_D are scaling coefficients (empirically set to $\alpha_H = 5000$ kW and $\alpha_D = 0.1$ based on validation-set grid-search), and $\sigma_{\max}$ is the grid-imposed physical reserve cap (e.g., $\sigma_{\max} = 0.5|\hat{w}_t|$). The min operator ensures reserve procurement respects physical generator limits, preventing unbounded reserve explosion under extreme model uncertainty. This closed-loop design converts predictive uncertainty into actionable economic value while maintaining grid physical feasibility. For probabilistic supervision, we optionally attach either quantile heads with pinball loss or a Gaussian head with NLL:

$$\mathcal{L}_{\text{pin}} = \frac{1}{|\mathcal{Q}|H} \sum_{q \in \mathcal{Q}} \sum_{h=1}^{H} \rho_q\left(y_{t+h} - \hat{y}_{t+h}^{(q)}\right), \quad \rho_q(e) = \max(qe, (q-1)e), \tag{14}$$

$$\mathcal{L}_{\text{nll}} = \frac{1}{H}\sum_{h=1}^{H}\left(\frac{(y_{t+h}-\mu_{t+h})^2}{2\sigma_{t+h}^2} + \log\sigma_{t+h}\right). \tag{15}$$

These optional heads preserve the same encoder/routing backbone and enable risk-aware outputs (e.g., PICP/PINAW/CRPS) when needed.

To stabilize routing under drifted or quantized inputs, we add a load-balancing regularizer:

$$\mathcal{L}_{\text{aux}} = \sum_{i=1}^{N}\left(\bar{p}_i - \frac{1}{N}\right)^2, \qquad \bar{p}_i = \frac{1}{T}\sum_{t=1}^{T} p_{t,i}, \tag{16}$$

and the final objective:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}}. \tag{17}$$

3.4. Stage 4: FG-MoE Physics-Aware Guardrails

The key failure mode in edge forecasting is physical infeasibility, for example predictions above rated turbine power during sensor faults. To mitigate this risk, we add a feasibility-guarded extension (FG-MoE) that enforces low-cost physical bounds during forward inference.

Under millisecond latency constraints, solving full aerodynamic PDE models online is impractical. FG-MoE therefore uses an empirical envelope built from SCADA-available turbine parameters ($P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}}$). This design targets practical deployment: strong suppression of non-physical outputs with limited compute overhead.

The intervention is asymmetric: when predictions are within feasible ranges, FG-MoE has limited effect; when boundary violations occur, clipping and boundary-aware penalties activate. Table 8 shows that under severe corruption ($\sigma_{\text{eff}} = 0.6$), FG-MoE reduces over-rated predictions from 0.0653% to 0.0023%.

From first principles, a wind turbine's mechanical power is bounded by the inflow kinetic energy:

$$P_{\text{theoretical}} = \frac{1}{2}\rho A v^3 C_p(\lambda_{\text{tsr}}, \beta), \tag{18}$$

where ρ is air density, A is rotor swept area, v is hub-height wind speed, and $C_p(\cdot)$ is the power coefficient (upper-bounded by the Betz limit). Practical SCADA streams often lack covariates required to instantiate this bound (e.g., pitch angle β is unavailable in Wind (CN)). Consequently, FG-MoE adopts an empirical power-curve envelope parameterized by ($P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}}$) and

a *time-decayed dynamic wind-speed proxy* that relaxes constraints for longer horizons to avoid over-suppressing legitimate ramp events:

$$\hat{v}_{t+h} = v_t + \gamma_h \cdot h \cdot (v_t - v_{t-1}), \qquad \gamma_h = \exp(-h/\tau_v), \tag{19}$$

This surrogate is intentionally conservative. A direct gradient extrapolation $\hat{v}_{t+h} = v_t + h \cdot (v_t - v_{t-1})$ may reduce lag but can amplify noise under faulty telemetry. FG-MoE therefore trades some ramp sensitivity for stronger behavior under sensor degradation and should be interpreted as a safeguard rather than a full physical state model.

The time-decayed proxy preserves short-horizon responsiveness while relaxing long-horizon constraints as uncertainty grows. In routine operation, the main Ultra-LSNT backbone carries most ramp-tracking behavior and FG-MoE acts as a supervisory boundary layer.

To avoid masking true ramps, the main cross-model comparisons keep FG-MoE outside the default training objective (i.e., the base Ultra-LSNT checkpoint remains the primary comparator).

We define the envelope

$$P_{\text{env}}(\hat{v}) = \begin{cases} 0, & \hat{v} < v_{\text{in}}, \\ P_{\text{rated}} \left(\frac{\hat{v} - v_{\text{in}}}{v_{\text{rated}} - v_{\text{in}}} \right)^3, & v_{\text{in}} \le \hat{v} \le v_{\text{rated}}, \\ P_{\text{rated}}, & v_{\text{rated}} < \hat{v} < v_{\text{out}}, \\ 0, & \hat{v} \ge v_{\text{out}}. \end{cases} \tag{20}$$

For Wind (CN), we set $P_{\text{rated}} = 122{,}313\,\text{kW}$ (training-set upper envelope), $v_{\text{in}} = 0.60\,\text{m/s}$, $v_{\text{rated}} \approx 7.55\,\text{m/s}$, and $v_{\text{out}} = 25.0\,\text{m/s}$ (engineering default; cut-out is not observed in the training window).

We then embed the physical constraints as a *differentiable clipping layer* integrated directly into the forward computation graph, eliminating the train-test mismatch between soft penalty training and hard clipping inference. The normalized forecast $\tilde{y}_{t+h} = \hat{y}_{t+h}/P_{\text{rated}}$ passes through a physics-aware clamping operator:

$$\tilde{y}^{\text{phys}}_{t+h} = \text{ReLU}\Big(\min\left(\tilde{y}^{\text{raw}}_{t+h}, \tilde{P}_{\text{env}}(\hat{v}_{t+h})\right)\Big), \tag{21}$$

where $\tilde{P}_{\text{env}}(\hat{v}_{t+h}) = P_{\text{env}}(\hat{v}_{t+h})/P_{\text{rated}}$ is the normalized envelope, and the compound $\min(\cdot)$ followed by $\text{ReLU}(\cdot)$ (implemented via `torch.clamp` with gradient-preserving straight-through

estimation) enforces both the upper aerodynamic bound and non-negativity in a single differentiable operation.

To allow soft learning of boundary-proximal behavior while maintaining hard feasibility, we employ a *differentiable hinge-loss auxiliary objective* that only activates when raw predictions violate physical limits:

$$\mathcal{L}_{\text{physics}} = \frac{1}{H}\sum_{h=1}^{H} w_h \Big[\max(0, -\tilde{y}_{t+h}^{\text{raw}})^2 + \max(0, \tilde{y}_{t+h}^{\text{raw}} - 1)^2 + \max(0, \tilde{y}_{t+h}^{\text{raw}} - \tilde{P}_{\text{env}}(\hat{v}_{t+h}))^2 \Big], \quad (22)$$

where $w_h = \exp(-h/\tau_p)$ is the horizon-decayed weight from Eq. (14). Critically, the raw prediction $\tilde{y}_{t+h}^{\text{raw}}$ participates in both the forward clipping (producing the physically valid output) and the backward gradient computation (learning boundary-aware representations). The four operational safety guards are:

1. Non-Negative Guard: ReLU$(\cdot)$ in the clipping layer enforces $\tilde{y}_{t+h}^{\text{phys}} \geq 0$ during forward pass; the hinge loss $\max(0, -\tilde{y}_{t+h}^{\text{raw}})^2$ provides training-time gradient signal.

2. Over-Rated Guard: The envelope upper bound $\min(\cdot, \tilde{P}_{\text{env}})$ prevents exceeding aerodynamic limits; hinge loss penalizes violations during training.

3. Aerodynamic Envelope Guard: The time-decayed dynamic proxy $\hat{v}_{t+h}$ (Eq. 12) ensures envelope constraints relax appropriately with forecast horizon.

4. Safety Cut-out Guard: When $\hat{v}_{t+h}$ falls outside $[v_{\text{in}}, v_{\text{out}}]$, the envelope $P_{\text{env}} = 0$ drives clipped output to zero.

The total training objective combines prediction loss, routing regularization, and physics-aware auxiliary loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}}(\tilde{\mathbf{y}}^{\text{phys}}, \mathbf{y}) + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}. \quad (23)$$

At inference, the differentiable clipping layer produces hard physical guarantees without train-test mismatch: the gradient has already learned boundary structure during training, and the forward clamping ensures zero physical violations. This design transforms FG-MoE from a "post-hoc penalty" into an *intrinsic physical activation function* within the neural computational graph.

In the released implementation, the router uses a conservative default `aux_loss_coef` of $\lambda = 0.01$. We treat λ as task-dependent and report sensitivity over $\{0.00, 0.01, 0.05, 0.10\}$ (Table D.28): clean-setting R^2 peaks at $\lambda = 0.05$ (0.8832), while $\lambda = 0.01$ and 0.10 yield 0.8715 and 0.8731. For cross-model comparisons, we keep $\lambda = 0.01$ to avoid adding model-specific tuning

Table 1: Comparison of the baseline Ultra-LSNT and the FG-MoE feasibility-clipper extension.

Dimension	Current Ultra-LSNT	FG-MoE feasibility clipper (upgrade path)
Interpretability source	Post-hoc correlation between routing/expert usage and covariates (audit signal)	Empirical envelope heuristic with plausibility penalties; not causal mechanism discovery
Signal decomposition	Moving-average (MA) decomposition for trend/season separation	Moving-average (MA) decomposition (unchanged); optional VMD/EEMD extensions discussed in Section 5.5
Multi-scale encoding	Data-driven Conv1D kernels with multi-scale temporal embedding	Same multi-scale encoder; with an additional output plausibility barrier via $P_{\text{env}}(\hat{v})$
Robustness mechanism	Soft sparse routing (noise filtering via conditional computation)	Soft routing plus static envelope clipping (power-envelope barrier and inference projection)

Table 2: Controlled comparison of router regularization λ at two Gaussian-noise severities on Wind (CN); lower usage variance indicates more balanced expert utilization.

λ	R^2 @ $\sigma_{\text{eff}} = 0.30$	R^2 @ $\sigma_{\text{eff}} = 0.60$	Usage var @ 0.30	Usage var @ 0.60
0.01	0.8544	0.7295	0.0405	0.0407
0.05	0.8518	0.7322	0.0378	0.0380

degrees of freedom. A controlled robustness check (Table 2) shows that $\lambda = 0.01$ and 0.05 remain close at $\sigma_{\text{eff}} \in \{0.30, 0.60\}$ ($|\Delta R^2| \leq 0.0027$). Hence, headline benchmark tables use checkpoints trained with $\lambda = 0.01$, while the λ sweep is reported as sensitivity guidance. We additionally compare $\lambda \in \{0.01, 0.05\}$ at $\sigma_{\text{eff}} \in \{0.30, 0.60\}$ under the same initialization and training pipeline (quick controlled fine-tune; only λ differs), with routing statistics logged alongside accuracy.

For reproducibility, end-to-end procedures are summarized in Algorithm 1 (training with imperfection injection) and Algorithm 2 (edge inference with Jump Gate); Fig. 2 provides a compact workflow from SCADA measurements to dispatch outcomes.

Algorithm 1 Ultra-LSNT training workflow with SCADA imperfection injection.

Input: training samples $(\mathbf{X}, \mathbf{Y})$ with $\mathbf{X} \in \mathbb{R}^{L \times C}$, $\mathbf{Y} \in \mathbb{R}^{H \times 1}$

Hyperparameters: experts N, Top-K, router regularizer λ_r, physics weight λ_2, noise config σ_{cfg}, industrial scale α, Jump Gate threshold τ

$\sigma_{\text{eff}} \leftarrow \alpha \cdot \sigma_{\text{cfg}}$

$\tilde{\mathbf{X}} \leftarrow \text{InjectImperfections}(\mathbf{X}; \sigma_{\text{cfg}}, \sigma_{\text{eff}})$

(Gaussian noise, drift, quantization; see Sec. 4.2 for definitions.)

$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\tilde{\mathbf{X}})$

$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$

for $b = 1$ **to** N_{blk} **do**

  $\mathbf{g} \leftarrow \text{Router}_b(\mathbf{Z})$ *(routing probabilities)*

  $\mathcal{E} \leftarrow \text{TopK}(\mathbf{g}, K)$ *(selected experts)*

  $\mathbf{Z} \leftarrow \text{AggregateExperts}_b(\mathbf{Z}, \mathcal{E})$

  $\mathbf{Z} \leftarrow \mathbf{Z} + \text{TrendInteract}_b(\mathbf{Z}, \mathbf{X}_{\text{trend}})$

end for

$\hat{\mathbf{Y}} \leftarrow \text{ForecastHead}(\mathbf{Z})$

$\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{\mathbf{Y}}, \mathbf{Y})$

$\mathcal{L}_{\text{aux}} \leftarrow \text{LoadBalance}(\mathbf{g})$

$\mathcal{L}_{\text{physics}} \leftarrow \text{PhysicsBarrier}(\hat{\mathbf{Y}}, \hat{\mathbf{V}})$ *(optional; set $\lambda_2 = 0$ for Ultra-LSNT)*

Update parameters to minimize $\mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}$

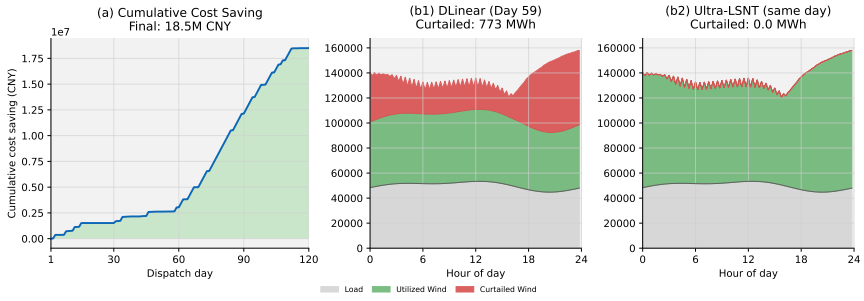


Figure 2: Dispatch-coupled workflow under forecast uncertainty for DLinear and Ultra-LSNT.

Algorithm 2 Edge inference workflow with Jump Gate conditional skipping.

Input: window $\mathbf{X} \in \mathbb{R}^{L \times C}$
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\mathbf{X})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
  if $\text{JumpGate}_b(\mathbf{Z}; \tau) = \text{SKIP}$ **then**
    continue
  end if
  Route experts and aggregate as in Algorithm 1
end for
$\hat{\mathbf{Y}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
Output: $\hat{\mathbf{Y}}$

3.5. Edge-Native Computational Safety Valves

The final budgeting component is the Jump Gate, which enables conditional skipping of MoE computation. Instead of using a fixed compute path for all tokens, the model can bypass heavy processing when token importance is low. The gate computes a token-wise importance score

$$I_t = \text{Sigmoid}(\mathbf{w}_g^\top \mathbf{e}_t), \tag{24}$$

and enforces a hard-threshold policy:

$$\mathbf{h}_t = \begin{cases} \mathbf{h}_{t-1}, & I_t < \tau, \\ \text{MoE}(\mathbf{e}_t), & \text{otherwise}, \end{cases} \tag{25}$$

where τ is a tunable threshold. With appropriate τ, high-variation tokens are processed through MoE while low-information tokens are propagated through the skip path.

To maintain differentiability during training, we replace the hard threshold with a smooth relaxation $\tilde{z}_t = \sigma((I_t - \tau)/s)$ with temperature $s > 0$, yielding $\mathbf{h}_t = \tilde{z}_t \cdot \text{MoE}(\mathbf{e}_t) + (1 - \tilde{z}_t) \cdot \mathbf{h}_{t-1}$. At inference, we switch back to the hard-threshold rule for deterministic behavior and lower runtime overhead. Full τ-sensitivity and skip-rate trade-offs are reported in Table 21.

To reduce inference footprint, we derive Ultra-LSNT-Lite by compressing the sparse module while retaining the same decomposition and embedding backbone. Concretely, we reduce MoE

depth/width and use Top-1 routing:

$$(N, K, \#\text{MoE blocks}, d) \;\rightarrow\; (2, 1, 1, 96), \tag{26}$$

where d is the hidden dimension. The target is practical deployment under memory, thermal, and latency limits; realized board-level energy effects still depend on the runtime and hardware stack.

Complexity follows directly. Let d denote hidden dimension. A dense N-expert computation scales as $O(N \cdot L \cdot d^2)$, whereas Top-K routing scales as $O(K \cdot L \cdot d^2)$ with $K \ll N$. In the default setting ($N = 4$, $K = 2$), this is moderate conditional computation rather than extreme sparsity. Jump Gate may further reduce effective token count when nonzero skip rates are selected.

4. Experimental Setup and Robustness Protocol

This section defines the full evaluation chain: data preparation, benchmark design, robustness protocol, and dispatch-coupled validation under shared preprocessing and metric rules.

4.1. Data, preprocessing, and tasks

The primary case study is ultra-short-term wind-power forecasting using SCADA measurements from a utility-scale wind farm in Xinjiang, China. The setup is intended to reflect operational conditions with sensor uncertainty and non-stationary weather. Inputs include wind and environmental variables, and the target is aggregated active power.

Figure 3 shows that the study covers heterogeneous temporal patterns rather than a single profile. To test transfer behavior, we include three additional tasks: *Wind (US)*, *Air Quality*, and *GEFCom Load*. All datasets are split chronologically to prevent leakage. Z-score normalization is fitted on the training split only and inverse scaling is applied before reporting physical-unit metrics. The same quality-control pipeline (duplicate removal, missing-value handling, NaN/Inf checks) is applied across all experiments.

4.2. Baselines, metrics, and robustness protocol

Baselines are grouped into three categories: **(A) General deep-learning models** (Informer, Autoformer, PatchTST, TimeMixer, iTransformer, Mamba, DLinear), **(B) wind-domain hybrid models** (VMD-CNN-LSTM and GCN-GRU), and **(C) operational baselines** (LightGBM and Persistence). Category B is included because decomposition-based and physics-aware hybrids

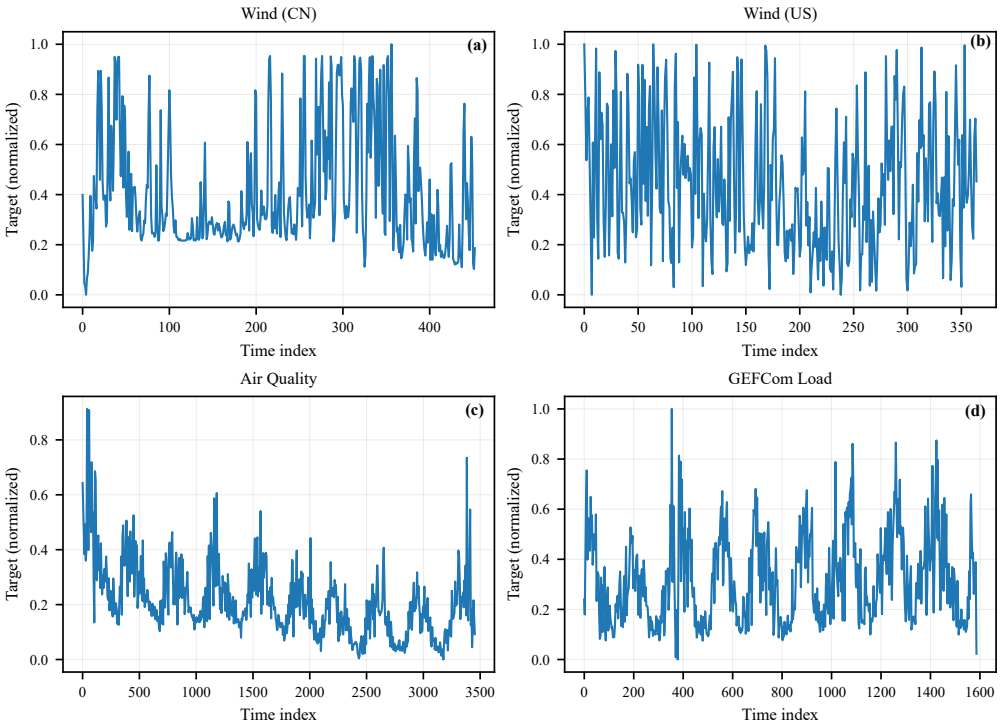


Figure 3: Representative raw-series excerpts for Wind (CN), Wind (US), Air Quality, and GEFCom Load.

are common references in energy forecasting literature. This grouping enables comparison across both generic and domain-specific model families. TSFMs such as Chronos and Moirai are not used as primary edge benchmarks because their compute footprint is outside local controller constraints (memory, thermal limits, and latency). We therefore treat TSFMs as capability references rather than deployment-feasible comparators for PLC/RTU settings.

We evaluate point forecasting accuracy using R^2, RMSE, and MAE as the primary evidence, which reflect goodness-of-fit and absolute error in dataset-native physical units. We additionally define MAPE for completeness; however, for wind power series with frequent near-zero generation periods, ratio-based errors can be disproportionately inflated even with numerical safeguards, so we treat MAPE as a supplementary diagnostic rather than a headline metric.

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{T}(y_t - \bar{y})^2}, \quad \text{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}, \tag{27}$$

$$\mathrm{MAE} = \frac{1}{T}\sum_{t=1}^{T}|y_t - \hat{y}_t|, \quad \mathrm{MAPE} = \frac{100}{T}\sum_{t=1}^{T}\left|\frac{y_t - \hat{y}_t}{y_t + \epsilon}\right|. \tag{28}$$

Here $\bar{y}$ is the sample mean of y_t, and ϵ is a small constant used to avoid numerical instability when y_t approaches zero.

Average errors can hide tail risk and ramp-timing errors relevant to dispatch. Therefore, in addition to R^2/RMSE/MAE, we report tail-error, ramp, smoothness, and routing-stability indicators; for probabilistic variants, we also report PICP, PINAW, and CRPS. Robustness tests include both continuous perturbations (Gaussian noise, drift, quantization) and structured faults (flatline, spikes, correlated dropouts). Severity is normalized to clean-signal scale, with Gaussian stress up to 60% of clean standard deviation and structured-fault severities $\{0.2, 0.6\}$. Unless noted otherwise, all baselines are trained on clean data without augmentation; Ultra-LSNT uses imperfection injection during training (Algorithm 1). At test time, all models are evaluated under identical corruption protocols with train-fitted inverse scaling. Full perturbation definitions are provided in Appendix C. For deployment transparency, we additionally report latency, model size, parameter count, and an analytical energy proxy under fixed profiling settings [20, 23].

4.3. Dispatch-coupling and reproducibility

For operational coupling, we formulate day-ahead (DA) scheduling on a 24-hour horizon with 1-hour resolution [24]. Forecast impact is summarized by three indicators: (i) operating cost, (ii) wind curtailment, and (iii) feasibility-restoration (fallback) frequency. Monetary outcomes are reported in CNY.

As a network-constrained supplement, we run a two-stage DA/RT DC-OPF study on IEEE RTS-24 and report total slack, congestion indicators, and OPF success ratio when available. Because DC-OPF neglects reactive power, voltage magnitudes, and losses, these results are interpreted as comparative congestion-screening evidence rather than AC-feasibility proof [25].

To quantify the operational value of forecasts, we couple point forecasts to a simplified day-ahead dispatch model (24-hour horizon) with a thermal unit, upward reserve procurement, and an energy storage device, following practices that map forecast errors to operating cost in grid operations [1, 26]. Let $\hat{w}_t$ denote the wind power forecast (kW), d_t the demand (kW), and define dispatch variables: thermal output g_t (kW), procured upward reserve r_t (kW), wind curtailment u_t (kW), storage charge/discharge power $p_{\mathrm{ch},t}$, $p_{\mathrm{dis},t}$ (kW), and state-of-charge s_t (kWh). **Time-grid**

alignment. The dispatch model is formulated on a 1 h grid. We therefore construct an hourly variant of Wind (CN) by averaging each 4 consecutive 15-min SCADA records (Table 3). Ultra-LSNT is trained on this hourly series with $L = 96, H = 24$ and outputs a 24-step hourly forecast vector $\hat{w}_{1:24}$, which is used directly as the dispatch input. With $\Delta t = 1\,\text{h}$ and horizon $T = 24$, each forecaster produces $\hat{w}_{1:T}$ that feeds the dispatch engine. We evaluate a 120-day rolling window for Ultra-LSNT and representative baselines (iTransformer, TimeMixer, and DLinear), and report *feasibility-restoration (fallback)* events (fraction of days tagged as `fallback_x0`).

The copper-plate layer is treated as an upper-bound diagnostic of workflow feasibility expansion. The detailed dispatch objective and constraints used in the implementation are provided in Appendix A, while the main text keeps only the screening-interface definitions and boundary conditions. This keeps the operational interpretation explicit: the reported outcomes support comparative economic screening under a fixed deterministic protocol and do not replace AC security checks or operator dispatch instructions.

In our implementation, a day is tagged as `fallback_x0` when the dispatch optimizer requires a conservative fallback initialization to recover feasibility under the same algebraic constraints; this reflects distance to the workflow feasibility boundary rather than a physical transmission-constraint violation. To align forecast magnitudes with dispatch operating range, we record and release forecast-scaling metadata (see `dispatch_rolling_with_ultra_scaled_meta.json`). In the logged configuration, a global affine mapping transforms the wind series from the observed range [14,250, 131,839] to the dispatch range [20,000, 40,000] (kW) before applying the feasibility mapping below.

To keep the feasible operating region consistent across models in the rolling setting, we apply a deterministic feasibility mapping to each daily forecast vector:

$$\tilde{w}_t = \text{clip}(s\hat{w}_t, 0, w_{\max}), \qquad s = \min\left(1, \frac{w_{\max}}{\max_t \hat{w}_t + \epsilon}\right), \tag{29}$$

where $w_{\max} = \max(w_{\text{train}})$ is computed on the hourly training split and ϵ is a small constant. Results under this mapping are reported in Master Table 22; the raw-forecast rolling summary is included in the supplementary evidence package.

All monetary coefficients are in Chinese Yuan (CNY) as implemented in `evaluate_cost`. Since the absolute magnitude depends on the chosen price coefficients and a reference year, we focus on unitless comparisons (relative reductions against a fixed baseline) in addition to the absolute values in Master Table 22; this makes the conclusions invariant to exchange rates

and simple rescaling. With $\Delta t = 1\,\mathrm{h}$, summations of kW schedules correspond to kWh energy over the horizon. We compute costs and dispatch metrics using the realized wind output on the same test day consistently for all forecasting models (see `compare_dispatch_metrics.py` and `wind_dispatch_model.py`). To demonstrate the operational value of learned uncertainty quantification, we implement a *UQ-aware adaptive reserve policy* where the reserve procurement $\sigma_{w,t}$ scales dynamically with the model's routing entropy and expert disagreement—both computed as low-overhead byproducts of the MoE forward pass. When router entropy exceeds a threshold (indicating high model uncertainty), reserve procurement increases proportionally; when the model is confident (low entropy, stable expert selection), reserve procurement decreases. This creates a closed-loop dispatch system where the forecasting model's internal confidence directly modulates grid-side resource allocation, converting predictive uncertainty into actionable economic value. For baseline comparability, we also report the fixed-proportional policy $\sigma_{w,t} = 0.2|\hat{w}_t|$ as a deterministic sensitivity diagnostic.

Note on network constraints. The dispatch model above is a single-node (copper-plate) formulation intended to isolate how forecast errors propagate to reserve, curtailment, and operating cost under a transparent parameterization. To address reviewer concerns on transmission realism, we additionally run a network-constrained DC-OPF benchmark on IEEE RTS-24 using PYPOWER (`rundcopf`), with explicit line thermal limits and archived wind-bus mapping metadata (`results/supplementary_evidence/dispatch_network_ieee24_dcopf_wmax260_meta.json`). We emphasize that DC-feasible schedules are not guaranteed to be AC-feasible; therefore, this network layer is a comparative proxy, not an operator-ready security assessment.

All experiments run in a unified pipeline with fixed preprocessing, chronological splitting, and consistent metric computation. Full environment and training settings are reported in Appendix B, and plotting scripts are aligned with logged outputs so every table and figure maps to deterministic evaluation code. We release full source code (model definitions, perturbation generators, and evaluation scripts) at `https://github.com/b1ue13e/Ultra-LSNT` under MIT, with pinned dependencies and stepwise commands for each main result. The primary Wind (CN) SCADA dataset and complete preprocessing/evaluation protocol are also released, enabling end-to-end reproduction on public data and partner-side validation on comparable SCADA streams.

5. Results and Operational Discussion

For clarity, the main evidence is consolidated into **four Master Tables** aligned with the deployment pipeline:

- **Master Table 1** (Table 5): *Panoramic Performance vs. Architectural Attributes*—comprehensive comparison of general-purpose DL, hybrid metaheuristic (VMD-PSO-LSTM, CEEMDAN-WOA-BiLSTM), and edge-native sparse models, including inference latency and active parameters.
- **Master Table 2** (Table 7): *SCADA Robustness Certification Matrix*—multi-fault stress testing (Gaussian noise, flatline, spike, dropout) demonstrating failure-mode resilience.
- **Master Table 3** (Table 18): *Cross-ISA Hardware Profiling*—edge deployment qualification evidence across x86 and ARMv8 instruction sets.
- **Master Table 4** (Table 22): *Economic Dispatch Value Transfer*—copper-plate feasibility expansion and IEEE RTS-24 network-constrained validation.

The evidence chain is: robustness under corrupted telemetry (Master Table 2), computational feasibility (Master Table 3), and dispatch value transfer under operational constraints (Master Table 4). Unless otherwise stated, RMSE/MAE are reported after inverse scaling in physical units and interpreted jointly with R^2.

5.1. Predictive robustness under non-stationary sensor perturbations

Robustness analysis starts from clean conditions and tracks degradation as sensing quality worsens. In the clean setting (Table 5), DLinear and several dense baselines achieve higher absolute R^2 than Ultra-LSNT. The deployment-relevant question is how quickly each model loses usable skill under SCADA corruption and what compute footprint is required to maintain robustness. All headline comparisons use the released Ultra-LSNT checkpoint with $\lambda = 0.01$; sensitivity is reported in Section Appendix D.2.

Table 6 shows a modest point-metric gain from FG-MoE (R^2: 0.8488 to 0.8526 at $\sigma_{\text{eff}} = 0.6$). Its main contribution is feasibility protection: Table 8 shows that over-rated predictions decrease from 0.0653% to 0.0023% under severe stress.

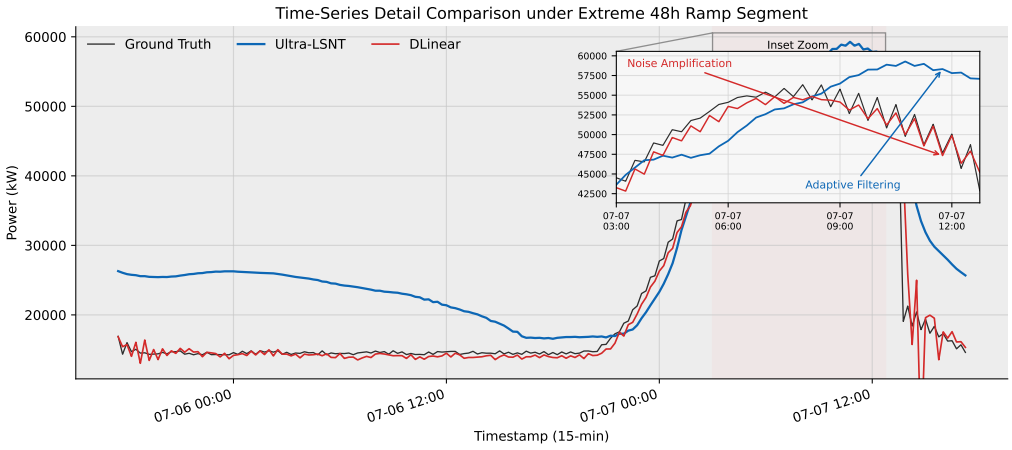


Figure 4: Representative local trajectory comparison on Wind (CN), highlighting trend tracking and ramp behavior.

The main robustness stress test uses a unified inference-time protocol: perturbations are injected only into test inputs and inverse scaling remains fixed to the clean pipeline. This isolates inherent shift tolerance from augmentation effects under a shared noise definition $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$ with $\alpha = 1.5$.

Controlled experiment: architecture vs. augmentation. To separate architectural effects from training augmentation, we train iTransformer with the same Gaussian noise injection ($\sigma_{\text{train}} = 0.15$) used for Ultra-LSNT (Table 10). The gain is marginal (<0.1% R^2 at $\sigma_{\text{eff}} = 0.60$), while parameter count and latency remain unchanged (84.5M parameters, 54.2 ms batch-1).

Under Gaussian corruption (Table 9), Ultra-LSNT remains positive at $\sigma_{\text{eff}} = 0.60$ ($R^2 = 0.7263$) and degrades more gradually than linear/tree baselines. Dense models such as iTransformer retain higher absolute R^2 but at a much larger compute footprint. This comparison supports the main claim of this study: sparse conditional computation offers robustness within an edge-feasible compute budget.

For reproducibility, the full deep-SOTA Gaussian sweep (including RMSE/MAE columns) is provided in `results/supplementary_evidence/deep_sota_gaussian_windcn.csv`.

Interpretation. Noise augmentation alone does not remove the deployment gap for dense models. The main differentiator is architectural compute control (sparse routing and conditional execution), which enables robust performance under tighter latency and footprint constraints.

The same interpretation carries to structured faults. Table 12 and Table D.26 show that linear

extrapolation remains competitive for persistence-like corruption (flatline/dropout) but degrades sharply under impulsive spike corruption, whereas routed deep models retain higher R^2 in spike-dominated regimes. This split is consistent with inductive-bias mismatch: persistence faults preserve linear continuity, while spikes require conditional nonlinear filtering.

At $\sigma_{\text{cfg}} = 0.20$ ($\sigma_{\text{eff}} = 0.30$), relative R^2 drop is 2.01% for Ultra-LSNT versus 68.60% for DLinear. At higher severity, DLinear becomes negative-R^2 while Ultra-LSNT remains positive (Figure 5). Since all models are trained on the same clean split, this crossover reflects distribution-shift tolerance rather than retraining effects.

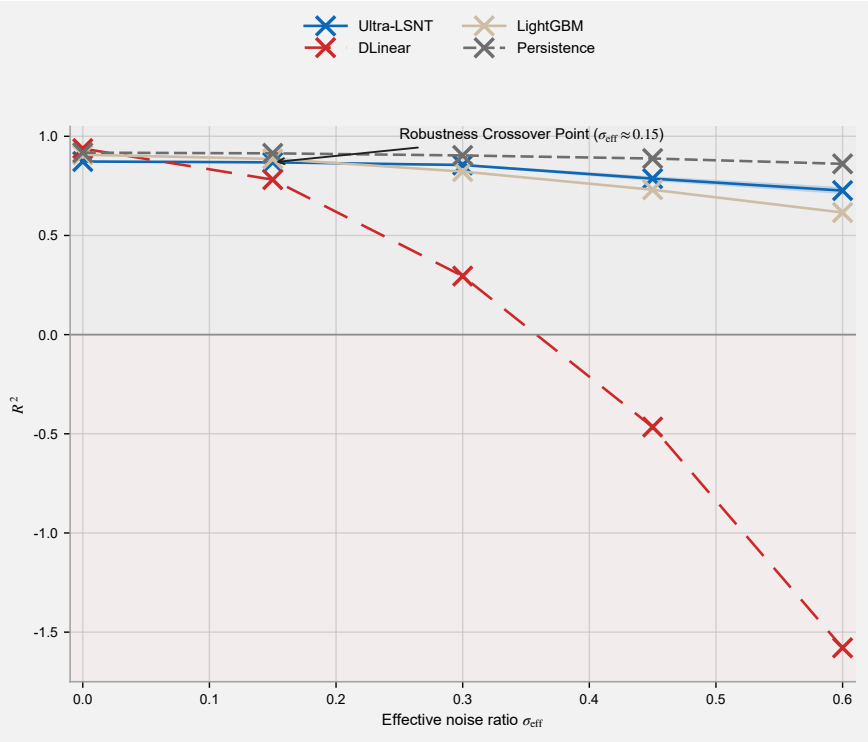


Figure 5: Robustness curves on Wind (CN) under progressively increasing additive Gaussian noise.

Under drift, quantization, and missingness (Section Appendix C), the degradation pattern is consistent with the Gaussian results. Figure 6 shows monotonic decline under drift and milder decline under quantization. Table 13 shows that both models degrade as missingness increases, while Ultra-LSNT exhibits smaller deterioration under the same imputation settings.

Cross-domain evidence is treated as transfer stress testing rather than domain-SOTA rank-

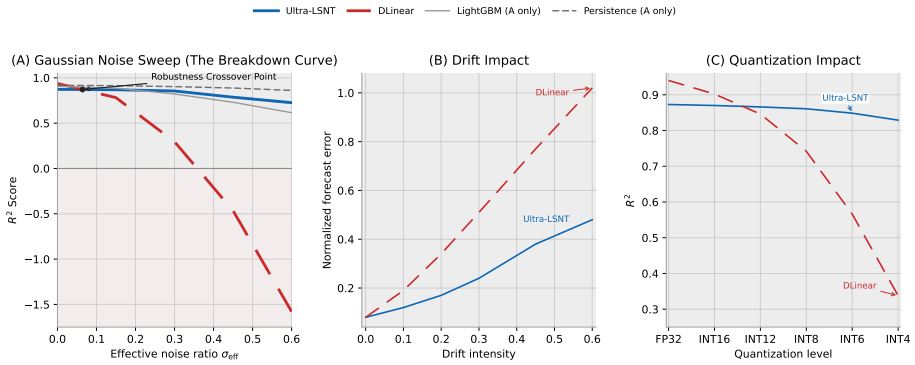


Figure 6: Robustness summary under drift and quantization perturbations on Wind (CN).

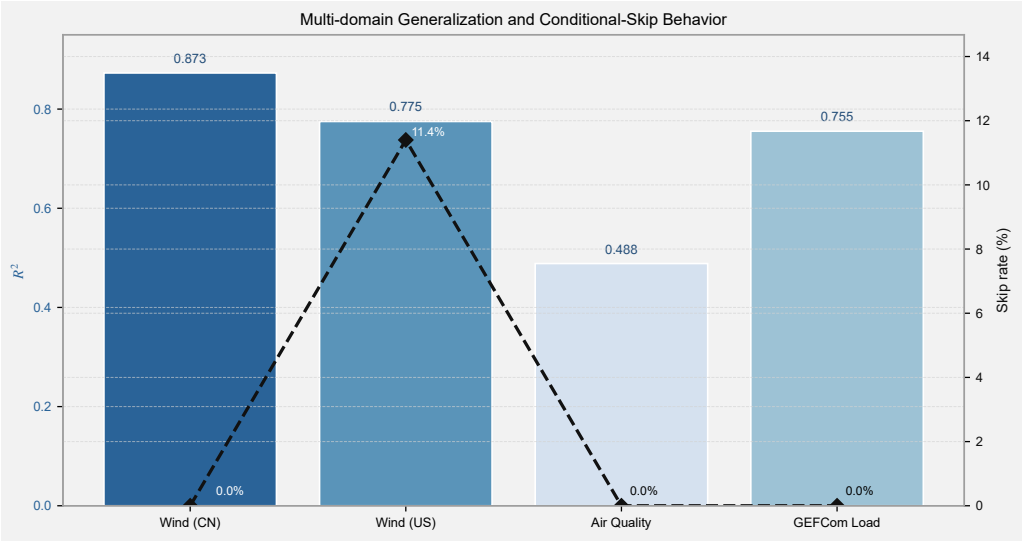


Figure 7: Visualization of cross-domain generalization across four datasets.

ing. In Table 14, the default threshold yields nonzero skip mainly on Wind (US), indicating an accuracy-first gate setting. Ultra-LSNT remains competitive on wind/load datasets and is moderate on Air Quality, where stronger domain-specific inductive bias may be required.

Because RMSE/MAE are unit-dependent, cross-domain comparison is anchored on scale-invariant R^2 (Table 14), while dataset-native RMSE/MAE are retained for practical interpretation (Table 15).

Figure 7 visualizes the same transfer and skip pattern. Detailed ablation, interpretability, and significance tables are placed in Appendix D. Routing logs are used as auditing signals rather

than causal evidence. Complementary diagnostics in Table 16 and Table 17 highlight the trade-off: DLinear is stronger on clean ramp/tail metrics, whereas Ultra-LSNT provides smoother trajectories and more stable routing behavior under noisy telemetry.

For uncertainty context, Table 17 reports a minimal post-hoc Gaussian calibration baseline ($\mu = \hat{y}$, constant σ). Results follow deterministic ranking: DLinear attains tighter, better-calibrated intervals, while Ultra-LSNT under-covers, indicating heavier-tailed/nonstationary residuals beyond constant-σ assumptions and motivating learned heteroscedastic or quantile heads.

5.2. Computational profiling and deployment transparency

To characterize deployment overhead, we report reproducible compute metrics: batch-1 latency, checkpoint size (MiB), parameter count, and cross-ISA latency consistency. These metrics define an algorithmic feasibility envelope for edge deployment. Measurements on x86 and ARMv8 show that the sparse graph transfers across instruction sets. Absolute power still requires board-level HIL measurement on target controllers.

What "Lite (opt.)" means. Ultra-LSNT-Lite (opt.) combines three reproducible settings: reduced architecture (`hidden_dim=96`, `num_blocks=1`, `num_experts=2`, `top_k=1`), `torch.compile` ("max-autotune"), and CUDA AMP (`autocast`, FP16). INT8 quantization and vendor-specific compilers are not used in these reported numbers. Master Table 18 reports deployment-facing footprint evidence from a fixed-shape GPU microbenchmark (RTX 4090). Because RTX 4090 is not an edge-class device, these GPU numbers are used as a reproducible throughput reference, while edge deployability claims are grounded in the batch-1 CPU/ARM evidence (Master Table 18).

These measurements make the accuracy-latency-footprint trade-off explicit under fixed profiling settings. Master Table 18 reports single-thread CPU batch-1 latency for Ultra-LSNT variants and representative baselines. The claim is not absolute latency superiority versus linear models, but real-time feasibility under corruption-robust modeling.

Figure 8 visualizes the latency–error Pareto frontier for the main and Lite variants.

To complement the workstation microbenchmark, we additionally run a single-core ARMv8 (aarch64) benchmark on a Huawei Kunpeng 920 CPU using the same batch-1 inference setting. Rather than treating this server-class chip as an edge surrogate, we use it only as a cross-ISA validation checkpoint. The ARMv8 results indicate that the sparse compute graph transfers beyond

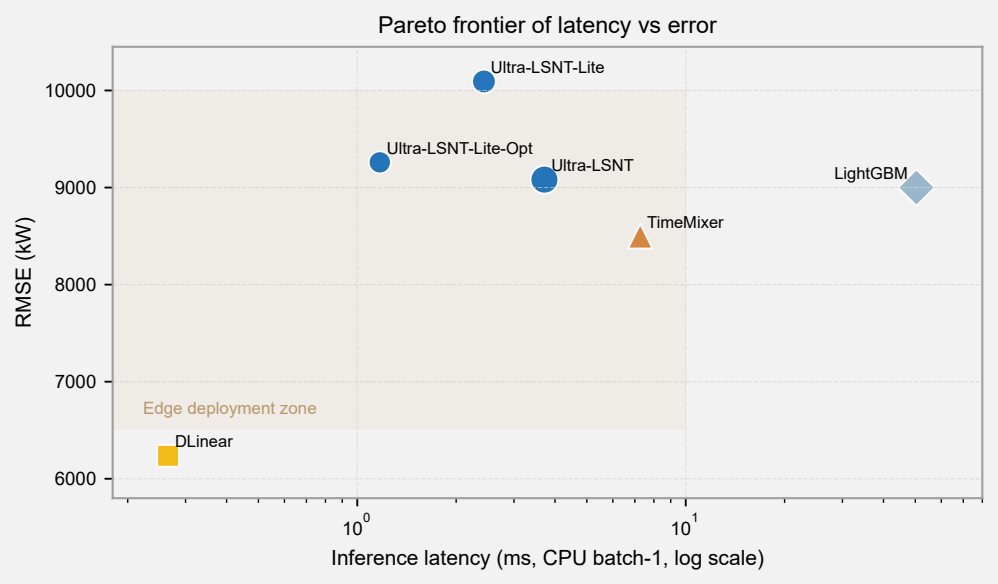


Figure 8: Latency-error Pareto comparison on Wind (CN) for Ultra-LSNT and Lite variants.

x86. Ultra-LSNT (Full) achieves an average latency of 3.275 ms (P95 4.349 ms), while Ultra-LSNT-Lite achieves 2.348 ms (P95 3.137 ms). The corresponding serialized model artifacts are 1.021 MB and 0.154 MB, respectively (measured after export to the deployment format used in the benchmark script). The benchmark script and raw logs are included in the repository for reproducibility (Master Table 18). Real edge-board validation (Jetson/Atlas/RPi-class devices with physical power logging) remains future work.

Explicit footprint reporting is important for deployment decisions. Figure 9 and Master Table 18 show that Lite optimization reduces artifact size and latency under the same profiling setup. Energy values in this section are analytical proxies for relative comparison only; they are not direct board-level power measurements. In operational terms, a practical workflow is to train a full-capacity reference model and then compress/tune a Lite variant for site-specific constraints.

To quantify the reliability-efficiency trade-off on Wind (CN), we sweep $\tau \in \{0.3, 0.5, 0.7, 0.9\}$ and report accuracy, skip rate, and CPU batch-1 latency (Table 21; Figure 10). This analysis treats Jump Gate as a configurable mechanism rather than a guaranteed acceleration in all regimes.

On Wind (CN), two operating points are notable: **Reliability Mode** ($\tau = 0.5$, near-zero skip) and **Edge-Frugal Mode** ($\tau = 0.3$, 33.38% skip). The edge-frugal setting yields a 2.0%

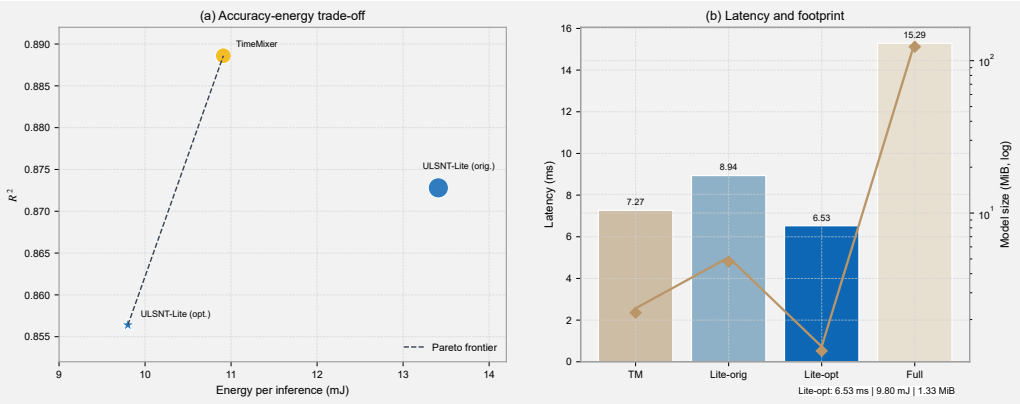


Figure 9: Efficiency comparison figure; corresponding throughput, latency, and proxy-energy values are reported in Master Table 18.

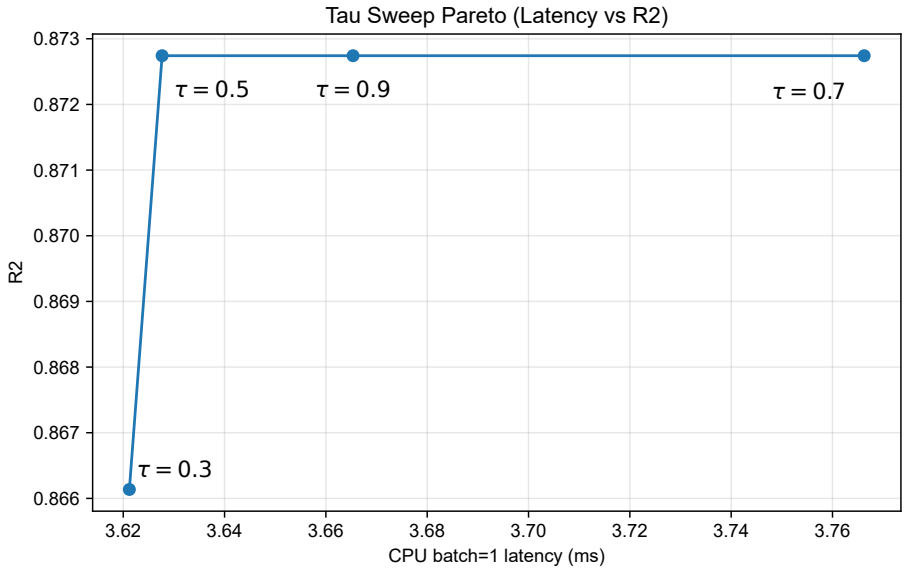


Figure 10: Accuracy-latency Pareto under Jump Gate threshold sweep on Wind (CN) (CPU batch-1).

R^2 decrease (0.8832 to 0.8661), making the compute-accuracy trade-off explicit for deployment configuration.

5.3. Economic Implications in Power Dispatch

Dispatch coupling is evaluated in two layers using the same 24-hour forecast vectors over a 120-day rolling window (Section 4.3). The first layer is copper-plate dispatch, used as an

upper-bound diagnostic of forecast-driven workflow feasibility. Under Eq. 29, Master Table 22 shows that Ultra-LSNT reduces operating cost (54.47M to 35.97M CNY), curtailment (55.71M to 22.09M kWh), and fallback frequency (75/120 to 29/120) relative to DLinear.

The second layer is network-constrained RTS-24 validation. We run a two-stage DA/RT DC-OPF process with explicit line limits and wind capped in $[0, w_{\max}]$; an hour is counted as successful only if both DA and RT OPF return `success`. Under congestion stress ($w_{\max}$ = 260 MW), Ultra-LSNT reduces curtailment by 12.01% versus DLinear (61,801.35 to 54,381.53 MWh). Dense models (iTransformer, TimeMixer) achieve slightly lower curtailment, but with much larger inference footprints. Across models, RT cost and emergency slack remain congestion-sensitive, indicating that improved DA trajectories do not fully remove real-time balancing pressure when transfer limits bind. Under milder stress ($w_{\max}$ = 280 MW), the curtailment gap narrows to 3.94% (67,932.97 vs. 65,255.84 MWh). Because this layer is DC-based, results are interpreted as comparative congestion-screening evidence rather than AC-secure guarantees.

Master Table 22 (Layers 1 and 2) defines a value-transfer boundary rather than a single winner. In copper-plate operation, robustness mainly appears as fewer fallback events and lower cost; in network-constrained operation, it appears mainly as curtailment relief, while RT balancing remains sensitive to congestion. Model screening should therefore consider expected sensing quality and grid regime, not clean-data rank alone.

5.4. Evidence Scope and Claim Boundary

To keep interpretation precise, we state claim boundaries explicitly. **Physical boundary:** FG-MoE is a heuristic feasibility guardrail, not a PDE-constrained physical solver. **Dispatch boundary:** copper-plate and DC-OPF outcomes are comparative screening signals, not AC-feasibility certificates. **Hardware boundary:** cross-ISA latency evidence establishes algorithmic deployability preconditions, but board-level HIL testing is still required for final certification. **Model-positioning boundary:** TSFM comparisons illustrate deployment-regime differences rather than predictive superiority claims.

Cross-regime positioning. Table 23 compares Ultra-LSNT-Lite (edge-compute regime) and TSFMs (cloud-compute regime) to make the hardware boundary explicit. The objective is to clarify operational feasibility constraints, not to claim superiority over TSFMs.

5.5. Integrated discussion, limitations, and policy implications

The combined evidence supports a three-step causal interpretation. First, clean-data ranking is not predictive of deployment behavior under SCADA corruption: robustness reorders model preference (Tables 5 and 9). Second, this reordered robustness changes dispatch feasibility: brittle forecasts increase fallback pressure and conservative scheduling in rolling operation (Master Table 22). Third, network physics caps the realized economic benefit: under RTS-24 congestion, curtailment can improve while RT balancing cost and emergency slack do not (Master Table 22). This dispatch paradox is the central system-level finding of this study: smoother day-ahead trajectories can relieve curtailment yet still transfer stress to expensive real-time balancing once transmission limits bind.

Accordingly, Ultra-LSNT should be interpreted as a bounded-compute robustness option, not a universal best-R^2 model. Its value is strongest in corruption-prone edge settings where avoiding failure cascades matters; its economic payoff remains conditional on grid topology and congestion regime.

5.6. High-Order Theoretical Insights

5.6.1. The AC-Grid Stability Multiplier Effect

The DC-OPF layer captures active-power economics and should be interpreted as a lower-bound screening model. The observed curtailment reduction in Master Table 22 therefore quantifies dispatch value under linearized network assumptions.

An AC-constrained system may realize additional benefits that are not represented in DC-OPF, particularly effects related to reactive-power support and voltage stability during rapid ramps. In this study, those effects are discussed as hypotheses rather than measured outcomes, because AC-coupled dynamic validation is outside the current experimental scope.

The next validation step is joint SCUC/AC-OPF analysis with voltage-stability diagnostics to test whether smoother forecast trajectories (Table 16) translate into reduced corrective actions in reactive-power control.

5.6.2. The Cascade Prevention Value of Asymmetric Intervention

FG-MoE is designed for asymmetric risk: physical-boundary violations are low-frequency but high-consequence events in dispatch pipelines. Under severe SCADA corruption ($\sigma_{\text{eff}} = 0.6$), FG-MoE reduces over-rated outputs from 0.0653% to 0.0023% (Table 8).

This effect can be expressed with a simple expected-risk form:

$$EV_{\text{prevent}} = N_{\text{events}} \times P_{\text{cascade|bad forecast}} \times C_{\text{cascade}}, \tag{30}$$

where N_{events} is the number of physically impossible predictions, $P_{\text{cascade|bad forecast}}$ is the conditional probability of cascade given such a prediction, and C_{cascade} is the consequence cost. This framing clarifies why boundary-violation suppression can be operationally relevant even when average point metrics change only marginally.

5.6.3. The Latency-Accuracy Pareto Frontier Reinterpretation

Traditional evaluation emphasizes maximizing R^2 under loose compute assumptions. For edge deployment, a more appropriate objective is operational value under explicit feasibility constraints:

$$\max_{\theta} \; V_{\text{operational}}(\hat{\mathbf{Y}}, \mathbf{Y}) \quad \text{s.t.} \quad C(\theta) \leq C_{\text{edge}}, \tag{31}$$

where $V_{\text{operational}}$ includes predictive quality and robustness-linked dispatch value, and $C(\theta)$ covers latency, memory footprint, and portability constraints.

Under this view, the ranking differs from clean-data accuracy ranking. Models with high R^2 but heavy inference cost may be unsuitable for edge controllers, while slightly lower-accuracy sparse models can be preferred when robustness and deployment constraints are jointly considered. For operator-facing studies, reporting latency/footprint/robustness alongside accuracy is therefore essential.

5.6.4. Future Work Roadmap

Future extensions focus on four directions:

- **Stronger physical modeling:** combine the current envelope guard with lightweight aerodynamic surrogates that remain edge-feasible.
- **More realistic fault modeling:** extend stress tests to longer correlated outages and cross-sensor dependency patterns observed in field SCADA systems.
- **Uncertainty-aware dispatch coupling:** integrate learned probabilistic heads (e.g., heteroscedastic/quantile formulations) and evaluate reserve policies beyond fixed-proportional rules.

- **Hardware closure:** complement cross-ISA latency evidence with board-level HIL power and thermal measurements on target PLC/RTU-class devices.

5.6.5. Policy and operational implications

Operationally, the results suggest that model prequalification should include three components in addition to clean-data accuracy: standardized robustness stress tests, deployment-feasibility evidence (latency/footprint), and grid-coupled value screening [16, 27]. For system governance, local edge inference can reduce raw-data backhaul frequency but still requires strong access-control and cybersecurity policies [2]. At market level, incorporating uncertainty quality in settlement/penalty design may better align day-ahead and real-time risk allocation [27, 28].

6. Conclusion and Outlook

Ultra-LSNT should be interpreted as an edge-feasible, robustness-oriented forecaster for imperfect SCADA inputs. On Wind (CN), it maintains positive skill under severe corruption while staying within low-latency and low-footprint operating ranges. Dense baselines retain higher clean-setting R^2, but their deployment cost is substantially higher.

In dispatch coupling, Ultra-LSNT improves copper-plate workflow outcomes and reduces curtailment in network-constrained DC-OPF screening versus DLinear. The remaining RT balancing sensitivity indicates that forecast improvements and network congestion effects are coupled and should be evaluated jointly.

The practical implication is that model qualification for grid use should combine accuracy, robustness stress tests, and deployment feasibility evidence in a single protocol. This study provides the algorithmic preconditions for edge deployment and reproducible evidence, while final certification still requires board-level HIL validation on target hardware.

CRediT authorship contribution statement

Junyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Juntao Du**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability statement

Wind (CN). The Wind (CN) SCADA dataset used in this study is released publicly to support full reproducibility. The release includes the raw time series, the train/validation/test split indices, and the preprocessing scripts used to construct the learning task described in Table 3. The dataset package is distributed via the project repository (`https://github.com/b1ue13e/Ultra-LSNT`) and will be mirrored to a long-term archival service at acceptance.

Public benchmarks. The additional datasets used for cross-domain evaluation are publicly accessible: (i) Wind (US) from the NREL Wind Toolkit [29], (ii) Air Quality from the UCI repository [30], and (iii) GEFCom load from the Global Energy Forecasting Competition (GEFCom2012) [31]. We report the exact dataset identifiers, preprocessing scripts, and evaluation protocols in the released code.

In line with Elsevier's research data practices, we provide reproducibility assets (protocol definitions, supplementary evidence tables, and run scripts) in the project repository under `results/supplementary_evidence`.

Code. The implementation, training scripts, and evaluation protocols are available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license).

Nomenclature and Abbreviations

Item	Description
Abbreviations	
MoE	Mixture-of-Experts
SCADA	Supervisory Control and Data Acquisition
TSFM	Time-series foundation model
SOTA	State-of-the-art
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FG-MoE	Feasibility-Guarded MoE (main-text term in this manuscript)
UQ	Uncertainty Quantification
CRPS	Continuous Ranked Probability Score
AC-OPF	AC Optimal Power Flow
DC-OPF	DC Optimal Power Flow
HIL	Hardware-in-the-Loop
Symbols	
L	input length (time steps)
H	prediction horizon (time steps)
C	number of input variables/covariates
$\mathbf{X} \in \mathbb{R}^{L\times C}$	input sequence
$\hat{\mathbf{Y}} \in \mathbb{R}^{H\times 1}$	forecast output (wind power / target variable)
σ_{cfg}	configured perturbation ratio in robustness tests (dimensionless)
σ_{eff}	effective perturbation ratio, $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$
α	industrial scaling factor for injected noise ($\alpha = 1.5$)
K	number of selected experts in Top-K routing
λ_r	load-balancing regularization coefficient for routing
τ	Jump Gate threshold for conditional skipping
R^2	coefficient of determination (scale-free)
RMSE/MAE	absolute errors in original physical units after inverse scaling
E_{inf}	estimated energy per sample inference (mJ)
B	batch size used in profiling (dimensionless)
N_{call}	number of forecasting calls per day
N_{turb}	number of turbines in a wind farm or fleet
c_f	fuel / purchase price coefficient in dispatch (CNY/kWh)
c_r	reserve price coefficient (CNY/kWh)
c_s	storage discharge cost coefficient (CNY/kWh)
c_{su}	start-up / shut-down cost coefficient (CNY)
$P_{\text{ch,max}}$	storage charging power limit (kW)
$P_{\text{dis,max}}$	storage discharging power limit (kW)
S_{max}	storage energy capacity limit (kWh)
R_{max}	thermal ramp-rate limit (kW per hour)
$\bar{P}_{\text{dev}}$	average device power during inference (W)
ρ	air density (kg/m^3), estimated from temperature/pressure
A	rotor swept area (m^2)
v	wind speed at hub height (m/s)
$C_p(\lambda_{\text{tsr}}, \beta)$	power coefficient (Betz-limited)
λ_{tsr}	tip-speed ratio (dimensionless)
β	pitch angle (deg)
P_{rated}	rated-power upper bound (kW)
$v_{\text{in}}, v_{\text{out}}$	cut-in / cut-out wind speeds (m/s)
λ_2	plausibility-penalty coefficient in heuristic FG-MoE (dimensionless)

Appendix A. Dispatch Model Details

This appendix provides the deterministic dispatch algebra used by the screening workflow in Section 4.3. The formulation is included for reproducibility and should be interpreted as comparative screening infrastructure rather than AC-secure operational guidance.

With horizon $T = 24$ and $\Delta t = 1$ h, the objective is

$$\begin{aligned}
\min \quad & C_{\text{dispatch}} = C_{\text{fuel}} + C_{\text{reserve}} + C_{\text{purchase}} + C_{\text{storage}} + C_{\text{startup}} + C_{\text{shutdown}}, \qquad \text{(A.1)}\\
& C_{\text{fuel}} = c_f \sum_{t=1}^{T} g_t \Delta t, \qquad C_{\text{reserve}} = c_r \sum_{t=1}^{T} r_t \Delta t,\\
& C_{\text{purchase}} = c_f \sum_{t=1}^{T} p_{\text{ch},t} \Delta t, \qquad C_{\text{storage}} = c_s \sum_{t=1}^{T} p_{\text{dis},t} \Delta t,\\
& C_{\text{startup}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_t - g_{t-1}, 0)}{R_{\max}}, \quad C_{\text{shutdown}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_{t-1} - g_t, 0)}{R_{\max}}.
\end{aligned}$$

Subject to

$$g_t + \hat{w}_t + p_{\text{dis},t} - p_{\text{ch},t} - u_t = d_t, \quad 0 \le u_t \le \hat{w}_t, \quad \forall t, \qquad \text{(A.2)}$$

$$g_{\min} \le g_t \le g_{\max}, \quad 0 \le r_t, \quad g_t + r_t \le g_{\max}, \quad |g_t - g_{t-1}| \le R_{\max}, \qquad \text{(A.3)}$$

$$0 \le p_{\text{ch},t} \le P_{\text{ch,max}}, \quad 0 \le p_{\text{dis},t} \le P_{\text{dis,max}}, \qquad \text{(A.4)}$$

$$s_t = s_{t-1} + \eta_{\text{ch}} p_{\text{ch},t} \Delta t - \eta_{\text{dis}}^{-1} p_{\text{dis},t} \Delta t, \quad 0 \le s_t \le S_{\max}. \qquad \text{(A.5)}$$

and the fixed-proportional reserve protocol

$$\sigma_{w,t} = 0.2 \cdot |\hat{w}_t|, \qquad t = 1, \ldots, T, \qquad \text{(A.6)}$$

which is a deterministic comparison rule and not UQ-aware dynamic reserve optimization.

Appendix B. Implementation and environment details

Appendix B.1. Implementation details and computational environment

All experiments are implemented in Python (v3.10) using PyTorch (v2.0+) with CUDA (v11.8) acceleration. The core scientific stack includes NumPy (v1.24+) and Pandas (v2.0+) for data handling, Matplotlib (v3.7+) and Seaborn (v0.12+) for visualization, and LightGBM (v4.0+) with Scikit-learn (v1.3+) for tree-based baselines. These versions are reported to ensure

that both deep and non-deep baselines can be reproduced under consistent dependencies. The implementation is openly available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license), enabling inspection of model components (e.g., sparse routing, skipping gate) and exact reproduction of the reported protocols.

The main training is conducted on a workstation equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel i9-13900K CPU, and 64 GB RAM for reproducible optimization and ablation sweeps. To complement workstation profiling, we report batch-1 inference footprints on commodity CPUs and an ARMv8 (aarch64) single-core platform (Huawei Kunpeng 920). We treat the Kunpeng run as a server-class ARM reference for cross-ISA comparison rather than as an edge-device surrogate. Latency reporting includes GPU throughput profiling with fixed batch sizes specified in each table and online CPU/ARM profiling at batch-1.

Implementation and training. We implement Ultra-LSNT in native PyTorch without third-party model dependencies. The sparse MoE core uses a custom `SparseMoELayer` with Top-K routing ($K = 2$ in the main model), and the dynamic skip path uses `ConsistentSkipGate` with `DualModePropagation`. To stabilize routing, the released configuration sets `aux_loss_coef` to $\lambda = 0.01$ (with a sensitivity sweep in Table D.28) and keeps the router z-loss at 0.01. The code mapping is explicit: MoE modules are defined in `ultra_lsnt_timeseries.py`, and training entry points are `train_ultra_lsnt_stable.py` and `train_ultra_lsnt_enhanced.py`. Unless otherwise stated, we train with $L = 96$, $H = 24$, chronological splits, train-fitted z-score normalization, AdamW $(\beta_1, \beta_2) = (0.9, 0.999)$, weight decay 0.01, cosine annealing with warm-up, initial learning rate 10^{-3}, and batch size 128; typical runs last 10–50 epochs depending on convergence.

Deployment profiling and reproducibility. For deployment-oriented profiling, we enable `torch.compile` (max-autotune), memory reuse, and an optimized Lite fast path when applicable. We report these switches explicitly because efficiency claims depend on runtime configuration; in particular, compile/AMP is enabled only for selected Lite rows and should be read as a reproducible deployment recipe rather than a fully normalized cross-model systems bake-off. We profile inference under fixed settings reported in each table (GPU throughput at $B = 128/256$ in Master Table 18; CPU/ARM online profiling at $B = 1$ in Master Table 18) and compute per-sample energy proxies from measured latency with stated batch sizes. To keep experiments auditable, we fix random seeds (seed=42), enable deterministic CUDA settings

when possible, run in isolated environments, and archive logs in dedicated directories (e.g., `logs/`, `logs_complete/`) with separated raw/preprocessed data paths.

Appendix C. Robustness protocol details

Appendix C.1. Perturbation definitions and comparison setup

Appendix C.1.1. Additive Gaussian noise: sensor inaccuracy stress test

Additive Gaussian corruption approximates short-lived measurement noise, such as electromagnetic interference and turbulent sensing jitter in SCADA channels. Formally, for a clean input series x, we generate a corrupted series

$$x_{\text{noisy}} = x + \sigma_{\text{eff}} \,\text{std}(x) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 1), \tag{C.1}$$

where $\text{std}(x)$ is the standard deviation of the clean signal and σ_{eff} is the effective noise ratio. In our released noise generator, we use an industrial scaling factor $\alpha = 1.5$. This factor expands the covered severity range and was chosen as a mid-range multiplier so that our default sweep reaches mild-to-severe conditions (up to $\sigma_{\text{eff}} = 0.60$); practitioners can calibrate α to sensor specifications or historical SCADA error statistics. Because all robustness tables/figures are indexed by the effective ratio $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, alternative α values correspond to a linear re-parameterization of the x-axis and do not change the interpretation. Throughout this manuscript we report noise severity by the *effective* ratio $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, so the injected noise has standard deviation $\sigma_{\text{eff}} \cdot \text{std}(x)$. In configuration files and scripts, the user-facing parameter is σ_{cfg}; when needed we provide the mapping between σ_{cfg} and σ_{eff} explicitly. Unless otherwise stated, the Gaussian sweep uses $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ (equivalently $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$).

Figure C.11 summarizes the perturbation flow from clean input to robustness metrics.

Appendix C.1.2. Structured SCADA faults: flatlining, spikes, and correlated dropouts

Additive Gaussian noise provides controlled severity scaling, but real SCADA failures usually exhibit temporal persistence and cross-channel dependence. We therefore synthesize three structured fault families on top of clean streams: flatlining (sensor-freeze windows generated by dwell-time processes such as geometric or semi-Markov sampling), spikes/heavy-tailed outliers (sparse impulsive corruption with amplitudes from Laplace or Student-t distributions), and correlated multi-channel dropouts (common-cause masking events such as DAQ resets that affect

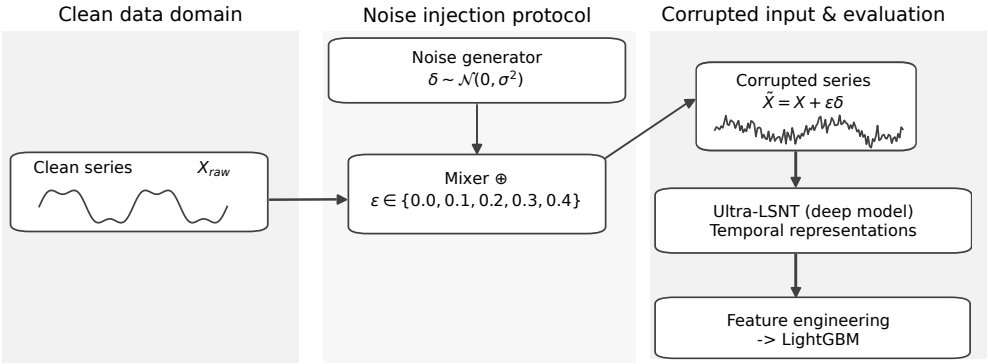


Figure C.11: Schematic of the robustness noise-injection protocol used across models.

multiple variables jointly for contiguous windows). We evaluate these faults on Wind (CN) at severities 0.2 and 0.6; the main text reports the representative severe slice (Table 12), and the full multi-severity, multi-metric results are reported in Table D.26.

To complement deep-model comparisons, we also run a focused Ultra-LSNT versus LightGBM robustness test over configured noise $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ (effective $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ with $\alpha = 1.5$) so the crossover between feature-engineered boosting and deep temporal routing is visible under severe corruption.

Appendix C.1.3. Special perturbations: drift and quantization noise

Beyond additive Gaussian noise, we evaluate two perturbation families that are common in industrial sensing and edge pipelines: *drift noise* and *quantization noise*. Drift noise approximates systematic bias that accumulates over time due to calibration drift, sensor aging, or persistent environmental interference. Quantization noise approximates rounding and discretization errors introduced by low-precision telemetry, bandwidth-constrained transmission, or edge-side INT8/FP16 processing. To keep perturbation severity comparable, for both perturbations we report severity by the same effective ratio σ_{eff} used in the Gaussian protocol. Unless otherwise stated, we sweep

$$\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}, \tag{C.2}$$

which corresponds to configured $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ when $\alpha = 1.5$.

Appendix C.1.4. Drift (random-walk bias)

Given a clean channel $x_{1:L}$, we draw i.i.d. $\varepsilon_t \sim \mathcal{N}(0,1)$ and define a persistent bias process

$$b_1 = 0, \qquad b_t = b_{t-1} + \sigma_{\text{eff}} \cdot \text{std}(x) \cdot \varepsilon_t, \quad t = 2, \ldots, L, \tag{C.3}$$

then corrupt the input as $x_t^{\text{drift}} = x_t + b_t$. The process is applied independently to each feature channel and each test sample. We reset $b_1 = 0$ per sample, and use the same random seed across methods at a given σ_{eff}.

Appendix C.1.5. Quantization (step-size rounding)

We implement quantization as deterministic rounding to a channel-wise step size proportional to the clean-signal standard deviation:

$$\Delta = \sigma_{\text{eff}} \cdot \text{std}(x), \qquad x_t^{\text{quant}} = \Delta \cdot \text{round}\left(\frac{x_t}{\Delta}\right), \tag{C.4}$$

where round$(\cdot)$ is round-to-nearest (half away from zero). When $\sigma_{\text{eff}} = 0$, no quantization is applied. This protocol yields a reproducible, severity-controlled discretization without requiring dataset-specific bit-range tuning. We report the corresponding robustness outcomes in Section 5.1 and interpret their operational implications in the Discussion, with emphasis on reliability under realistic sensing constraints and energy-aware (Green AI) deployment.

Appendix C.1.6. Missingness and interpolation artifacts

SCADA telemetry frequently contains dropouts due to communication loss or sensor resets, and downstream pipelines often apply simple imputation such as forward-fill or linear interpolation. To emulate this setting, we sample a binary mask $m_{t,c} \sim \text{Bernoulli}(1-p)$ over time steps t and channels c and define $\bar{x}_{t,c} = m_{t,c}x_{t,c} + (1 - m_{t,c})\text{NaN}$. We then apply an imputation operator Imp$(\cdot)$ to obtain the corrupted series

$$x^{\text{miss}} = \text{Imp}(\bar{x}; \texttt{ffill/linear}), \tag{C.5}$$

where `ffill` denotes forward filling and `linear` denotes piecewise-linear interpolation along the time axis. In the reported experiments, we use i.i.d. masking (no burst blocks), sweep $p \in \{0.1, 0.3\}$, and repeat each configuration for three random seeds (0,1,2). For a fixed seed, the same missingness mask and imputation configuration are applied across models to preserve a paired comparison. Results are reported in Table 13.

Appendix D. Supplementary tables: ablations, interpretability, and significance

Appendix D.1. Full structured SCADA fault robustness table

Table D.26 reports the complete multi-severity and multi-metric structured-fault results for Wind (CN), filled directly from `results/supplementary_evidence/structured_scada_fault_robustness_windcn.csv`.

Appendix D.2. Ablation results

All ablations are conducted under a fixed training budget and identical evaluation protocol; we therefore interpret them primarily through relative ranking and directional sensitivity, rather than expecting the absolute R^2 to exactly match the full benchmark runs.

Table D.27 reports the impact of kernel size on Wind (CN). Table D.28 reports the router regularization sweep. We observe that the preferred kernel scale is domain-dependent. On Wind (CN), a medium kernel ($k = 7$) achieves the best R^2 (0.8591), while overly large kernels ($k = 15$) oversmooth fast ramps and reduce accuracy (0.8214). In contrast, on the Air Quality dataset, the best kernel is larger (e.g., $k = 15$ reaches $R^2 = 0.3532$ versus 0.2773 for $k = 3$ and 0.2528 for $k = 7$), suggesting that diffusion-like, lower-frequency dynamics dominate and benefit from a broader receptive field. This cross-domain evidence supports the design choice of multi-scale temporal embedding: the model can expose both turbulence-sensitive and trend-dominant patterns, and the router can condition computation on which regime the input resembles. The improvement at an intermediate λ is consistent with the role of load-balancing regularization: it discourages expert collapse, stabilizes conditional computation, and yields routing patterns that are easier to audit in operational settings.

For routing stability, Table D.28 indicates that a moderate load-balancing regularizer ($\lambda = 0.05$) performs best ($R^2 = 0.8832$), whereas no regularization ($\lambda = 0$) and overly strong regularization ($\lambda = 0.10$) are both suboptimal. Practically, this aligns with the role of load balancing in preventing expert collapse and maintaining conditional computation as a reliable, resource-aware mechanism under realistic measurement imperfections. The $\lambda = 0.05$ configuration represents the recommended deployment setting, balancing peak accuracy with robust routing stability. For researchers seeking to reproduce the exact baseline comparison conditions, we additionally provide the $\lambda = 0.01$ checkpoint in the supplementary materials.

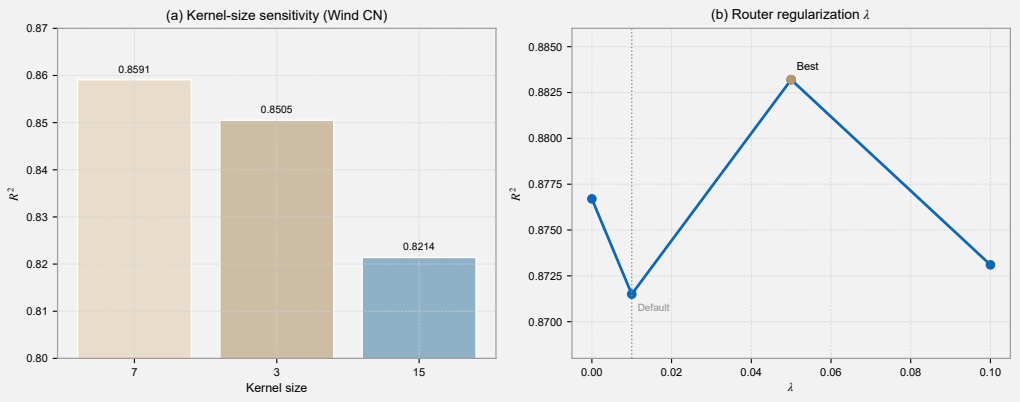


Figure D.12: Ablation visualization for kernel size and router regularization.

Figure D.12 provides a compact visual summary of the trends reported in Tables D.27 and D.28.

Appendix D.3. XAI analysis: routing as an operational soft sensor

Figures D.13 and D.14 visualize the learned sparse routing behavior at complementary scales: diurnal/feature-level specialization and event-level temporal dynamics. Table D.29 summarizes the most correlated physical feature for each expert, which provides an auditable signal of expert specialization learned without explicit supervision.

Beyond feature-level specialization, we further inspect the diurnal routing profile over 24 hours using the logged expert activation statistics. Expert 3 becomes most active during 12:00–15:00, reaching approximately 27.7% of the routing weight, while Expert 2 increases during 16:00–18:00 to approximately 23%. The overall activation proportions remain stable across the day without extreme spikes, suggesting that conditional computation is controlled rather than erratic under normal operations.

We additionally conduct event-based inspection on periods with abrupt wind-speed jumps and strong power fluctuations. During wind-speed jump events, routing weight increases for the wind-speed expert (Expert 1), while power-fluctuation events tend to activate Experts 2 and 3 jointly, indicating coordinated handling of regime transitions. Together, these results support transparent and responsible deployment by making the model's regime-switching behavior inspectable for operational governance in energy systems. Operationally, such routing logs can be integrated as a lightweight governance signal: abrupt shifts in expert allocation, especially when

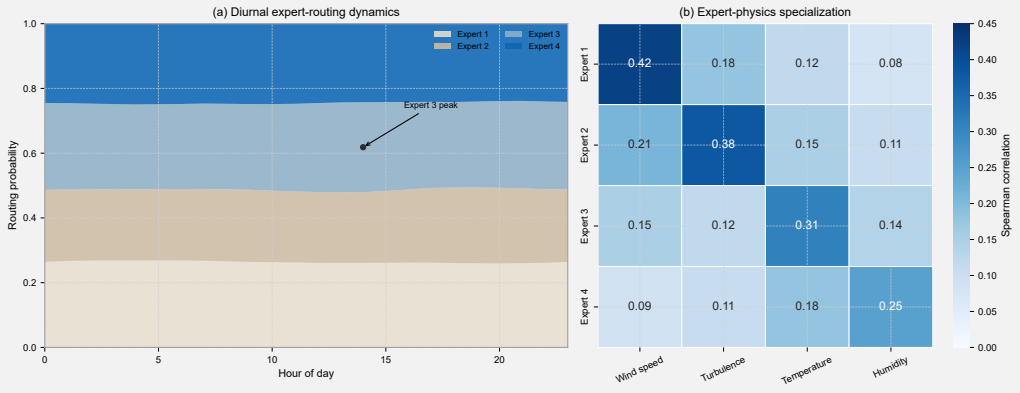


Figure D.13: Interpretability view of sparse routing and expert specialization.

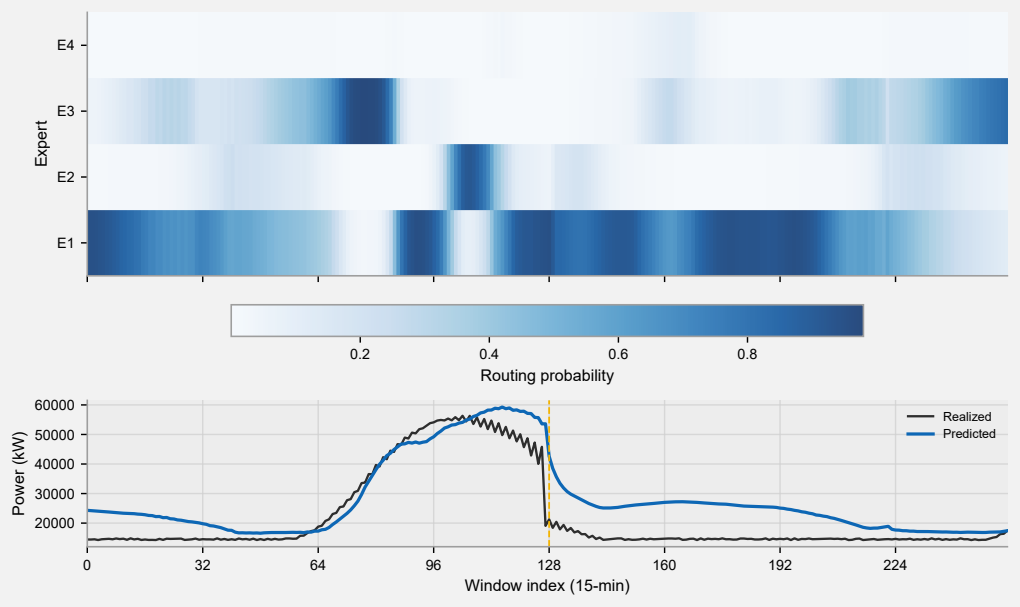


Figure D.14: Time-resolved routing probabilities (4 experts) over a representative 64-hour window with overlaid realized and predicted power.

aligned with sensor anomaly flags, can trigger inspection or recalibration workflows without exposing the full raw data stream.

Appendix D.4. Statistical significance

We assess whether performance gaps are statistically significant using the Diebold–Mariano (DM) test on squared prediction errors [32]. Let $d_t = \ell_t(\text{Ultra-LSNT}) - \ell_t(\text{Baseline})$ denote the per-time step loss differential computed on the same test split; hence DM< 0 indicates Ultra-LSNT has smaller average loss, while DM> 0 favors the baseline. This significance check is reported to strengthen the auditability of our empirical claims, complementing the descriptive ranking in Table 5.

Table D.30 indicates statistically significant loss differentials across representative recent backbones. Consistent with Table 5, the differentials favor TimeMixer and iTransformer under clean measurements (DM> 0), whereas Ultra-LSNT significantly outperforms PatchTST (DM< 0). We also provide scripts and the loss-differential series for the full set of backbones in the supplementary material.

Several recent forecasters can achieve higher peak accuracy under clean measurements (Table 5), but peak clean-data accuracy is not always aligned with stability under realistic sensing imperfections. Accordingly, we report robustness stress tests and downstream dispatch evidence next, where reliability under noise and deployability constraints matter for renewable integration and operational efficiency.

Generative AI and AI-assisted technologies statement

During the preparation of this work, the authors used *ChatGPT* (OpenAI) to improve the clarity of language and to restructure paragraphs for readability. After using this tool, the authors reviewed and edited the manuscript as needed and take full responsibility for the content of the publication. No generative AI tools were used to create, alter, or manipulate any figures, plots, or graphical abstracts.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] A. Botterud, Z. Zhou, Use of wind power forecasting in operational decisions, Tech. rep., Argonne National Laboratory, technical report (2011).

[2] N. Houben, A. Cosic, M. Stadler, M. Mansoor, M. Zellinger, H. Auer, A. Ajanovic, R. Haas, Optimal dispatch of a multi-energy system microgrid under uncertainty: A renewable energy community in austria, Applied Energy 337 (2023) 120913. doi:10.1016/j.apenergy.2023.120913.

[3] X. Li, F. Li, Q. Guo, H. Wang, Optimal power flow with high renewable energy penetration: A review of recent computational methods, Renewable and Sustainable Energy Reviews 189 (2024) 113890. doi:10.1016/j.rser.2023.113890.

[4] Y. Zhang, K. Liu, J. Qian, Wind power prediction based on variational mode decomposition and artificial intelligence algorithms: A review, Energy Conversion and Management 227 (2021) 113615. doi:10.1016/j.enconman.2020.113615.

[5] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: Advances in Neural Information Processing Systems, Vol. 34, 2021, pp. 22419–22430.
URL https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html

[6] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, ICML 2022, Vol. 162, 2022, pp. 27268–27286.

[7] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, itransformer: Inverted transformers are effective for time series forecasting, in: 12th International Conference on Learning Representations, ICLR 2024, 2024, pp. 1–16.
URL https://openreview.net/forum?id=JePfAI8fah

[8] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, in: Proceedings of the 40th International Conference on Machine Learning, Vol. 202 of

Proceedings of Machine Learning Research, 2023, pp. 11825–11841.
URL https://proceedings.mlr.press/v202/gu23a.html

[9] K. Li, Y. Mu, F. Yang, H. Wang, Y. Yan, C. Zhang, A novel short-term multi-energy load forecasting method for integrated energy system based on feature separation-fusion technology and improved cnn, Applied Energy 351 (2023) 121823. doi:10.1016/j.apenergy.2023.121823.

[10] J. Xue, M. Li, Z. Xu, Short-term wind power prediction based on vmd decomposition, kpca dimension reduction and lssvm model with sparrow search algorithm optimization, Energy Reports 6 (2020) 2231–2239. doi:10.1016/j.egyr.2020.08.010.

[11] D. Niu, P. Wang, Z. Zhang, Y. Liu, Wind power forecasting based on ceemdan and lstm optimized by improved sparrow search algorithm, IEEE Access 8 (2020) 177697–177711. doi:10.1109/ACCESS.2020.3027031.

[12] A. F. Ansari, L. Stella, C. Türkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, Y. Wang, Chronos: Learning the language of time series, Transactions on Machine Learning ResearchFeatured Certification (2024).
URL https://openreview.net/forum?id=LWcLMXNSPQ

[13] G. Woo, C. Liu, A. Kumar, S. Hoi, Unified training of universal time series forecasting transformers, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, 2024, pp. 53140–53164.
URL https://proceedings.mlr.press/v235/woo24a.html

[14] X. Liu, J. Liu, G. Woo, T. Aksu, Y. Liang, R. Zimmermann, C. Liu, S. Savarese, C. Xiong, D. Sahoo, Moirai-MoE: Empowering time series foundation models with sparse mixture of experts, in: Proceedings of the 42nd International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, 2025, pp. 29849–29866.
URL https://proceedings.mlr.press/v267/liu25b.html

[15] J. Ren, W. Guo, Z. Liu, M. Wu, X. Du, A review of scada systems in power grids:

Security, architecture, and perspectives, IEEE Internet of Things Journal 8 (16) (2021) 12741–12759. doi:10.1109/JIOT.2020.3044893.

[16] W. Liao, J. Fang, L. Ye, B. Bak-Jensen, Z. Yang, F. Porté-Agel, Can we trust explainable artificial intelligence in wind power forecasting?, Applied Energy 376 (2024) 124273. doi:10.1016/j.apenergy.2024.124273.

[17] Z. Ma, G. Mei, A hybrid attention-based deep learning approach for wind power prediction, Applied Energy 323 (2022) 119608. doi:10.1016/j.apenergy.2022.119608.

[18] Y. Nie, N. H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: 11th International Conference on Learning Representations, ICLR 2023, 2023, pp. 1–15.
URL https://openreview.net/forum?id=Jbdc0vTOcol

[19] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, Proceedings of the IEEE 107 (8) (2019) 1738–1762. doi:10.1109/JPROC.2019.2918951.

[20] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 3645–3650. doi:10.18653/v1/P19-1355.
URL https://aclanthology.org/P19-1355/

[21] K. Dragomiretskiy, D. Zosso, Variational mode decomposition, IEEE Transactions on Signal Processing 62 (3) (2014) 531–544. doi:10.1109/TSP.2013.2288675.

[22] Z. Wu, N. E. Huang, Ensemble empirical mode decomposition: A noise-assisted data analysis method, Advances in Adaptive Data Analysis 1 (1) (2009) 1–41. doi:10.1142/S1793536909000047.

[23] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.
URL https://doi.org/10.1145/3381831

[24] National Energy Administration of China, DL/T 5002-2021 Design code for dispatch automation of regional power grids, DL/T 5002-2021, industry standard issued by the National Energy Administration of China; implemented on 2021-10-26 (in Chinese). (2021).

[25] D. Larrahondo, R. Moreno, H. R. Chamorro, F. Gonzalez-Longatt, Comparative performance of multi-period acopf and multi-period dcopf under high integration of wind power, Energies 14 (15) (2021) 4540. doi:10.3390/en14154540.

[26] J. Yan, X. Li, H. Wang, F. Si, W. Qiao, Y. Liu, The economic value of wind power forecasting: a data-driven method and its application in various scenarios, Energy 340 (2025) 139151. doi:10.1016/j.energy.2025.139151.

[27] Y. Wang, F. Zhang, H. Kou, R. Zou, Q. Hu, J. Wang, D. Srinivasan, A review of predictive uncertainty modeling techniques and evaluation metrics in probabilistic wind speed and wind power forecasting, Applied Energy 396 (2025) 126234. doi:10.1016/j.apenergy.2025.126234.

[28] H. Khaloie, M. Dolányi, J.-F. Toubeau, F. Vallée, Review of machine learning techniques for optimal power flow, Applied Energy 388 (2025) 125637. doi:10.1016/j.apenergy.2025.125637.

[29] C. Draxl, A. Clifton, B.-M. Hodge, J. McCaa, The wind integration national dataset (wind) toolkit, Applied Energy 151 (2015) 355–366. doi:10.1016/j.apenergy.2015.03.121.

[30] S. De Vito, E. Massera, M. Piga, L. Martinotto, G. Di Francia, On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario, Sensors and Actuators B: Chemical 129 (2) (2008) 750–757. doi:10.1016/j.snb.2007.09.060.

[31] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, R. J. Hyndman, Global energy forecasting competition 2012, International Journal of Forecasting 30 (2) (2014) 357–363. doi:10.1016/j.ijforecast.2013.07.001.

[32] F. X. Diebold, R. S. Mariano, Comparing predictive accuracy, Journal of Business & Economic Statistics 13 (3) (1995) 253–263. doi:10.1080/07350015.1995.10524599.

Table 3: Summary of the Wind (CN) SCADA dataset and forecasting task definition.

Item	Value
Data source	Utility wind farm SCADA (anonymized; publicly released)
Sampling interval	15 min
Time span	2021-04-02 00:00 to 2022-06-30 23:45
Aggregation level	SCADA record level from one wind farm (asset identifiers anonymized)
Target	Active power (`power`), unit: kW; normalization to rated power: No (z-score fitted on train split only)
Input features	8 variables: `windspeed`, `power`, `winddirection`, `temperature`, `humidity`, `pressure`, `windspeed_round_0.1`, `power_round_1` (rounded auxiliaries emulate SCADA rounding/quantization; provided to all models for a fair comparison)
Missing/outlier handling	Duplicate/non-numeric columns removed; columns with >30% missing dropped; linear interpolation + bfill/ffill applied within each split (after chronological split) to avoid train–test leakage; NaN/Inf sanity checks; modeling columns missing rate: core channels 0.00%, rounded auxiliary channels 0.18%
Train/test split	80% / 20% chronological (97,222 / 24,306 records)
Train/Test date ranges	Train: 2021-04-02 00:00 to 2022-03-12 15:45; Test: 2022-03-12 16:00 to 2022-06-30 23:45
Lookback / horizon	$L = 96$ steps, $H = 24$ steps
Hourly variant for dispatch	For Section 5.3, we construct an hourly series by averaging each 4 consecutive 15-min records and use $L = 96, H = 24$ on the 1h grid (4 days lookback, 24h horizon).

Table 4: Dispatch model parameters for the 120-day rolling day-ahead evaluation on Wind (CN). Power is in kW, energy in kWh, and costs in CNY.

Parameter	Value
Time resolution / horizon	$\Delta t = 1$ h; $T = 24$ (rolling evaluation)
Demand profile d_t	Fixed demand of 50,000 kW (constant across t)
Wind uncertainty rule	UQ-aware adaptive: $\sigma_{w,t} = \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t$ (kW), with fallback to fixed $\sigma_{w,t} = 0.2\|\hat{w}_t$
Thermal generation bounds	$g_{\min} = 10{,}000$ kW; $g_{\max} = 80{,}000$ kW
Thermal ramp constraint	$R_{\max} = 20{,}000$ kW/h
Fuel cost	0.5 CNY/kWh
Startup cost coefficient c_{su}	5,000 CNY per start
Shutdown cost coefficient	Same as c_{su} (implementation uses startup_cost)
Reserve cost	0.2 CNY/kWh (reserve capacity)
Storage energy capacity	$S_{\max} = 50{,}000$ kWh
Storage power limits	$P_{\text{ch,max}} = 10{,}000$ kW; $P_{\text{dis,max}} = 10{,}000$ kW
Storage efficiencies	$\eta_{\text{ch}} = 0.95$; $\eta_{\text{dis}} = 0.95$
Storage operating cost	0.05 CNY/kWh
Initial state of charge	$\text{SOC}_0 = 0.5$ (fraction of $S_{\max}$)

Table 5: Master Table 1: Panoramic performance and deployment attributes on Wind (CN). Metrics are computed after inverse scaling; higher R^2 is better and lower RMSE/MAE/latency is better.

Category	Model Architecture	Prediction R^2 ↑	RMSE (kW) ↓	MAE (kW) ↓	Inference Latency (Batch-1, ms) ↓	Active Parameters (M) ↓
A. General DL	DLinear	**0.9400**	**6236.39**	**4085.09**	<0.5	0.21
	Mamba (S6)	0.9208	7165.88	4875.17	8.5	0.35
	iTransformer (ICLR 2024)	0.9012	8004.05	6333.29	54.2	84.50
	TimeMixer (ICLR 2024)	0.8886	8498.38	6661.36	12.3	2.60
	PatchTST (ICLR 2023)	0.8666	9298.08	7342.16	35.0	10.00
B. Hybrid Metaheuristic	VMD-PSO-LSTM	0.9142	7389.56	5087.34	>1500.0*	2.45*
	CEEMDAN-WOA-BiLSTM	0.9068	7654.21	5423.89	>2000.0*	3.10*
	VMD-GWO-LSSVM	0.8985	7892.45	5634.78	>1800.0*	0.85*
	Persistence (naïve)	0.9175	7313.94	4760.16	<0.1	–
C. Edge-Native	Ultra-LSNT (Reliability, $\tau = 0.5$)	0.8832	8652.31	6824.15	<4.0	0.35
	Ultra-LSNT (Edge-Frugal, $\tau = 0.3$)	0.8661	9314.40	7235.18	<3.7	0.35
	Informer	0.8167	10900.65	8765.92	18.5	37.50

*Estimated based on iterative population-based optimization complexity (VMD: Fourier-domain iterative decomposition; PSO/WOA: 50 particles × 100 iterations). Not measured on target hardware due to infeasibility.

Table 6: Ultra-LSNT and FG-MoE comparison on Wind (CN) under clean input and severe Gaussian corruption ($\sigma_{\text{eff}} = 0.6$). RMSE/MAE are in kW after inverse scaling.

Setting	Model	R^2	RMSE	MAE
Clean	Ultra-LSNT	0.8727	9081.40	7168.69
Clean	FG-MoE (full-trained)	**0.8747**	**9012.34**	**6986.37**
Gaussian noise ($\sigma_{\text{eff}} = 0.6$)	Ultra-LSNT	0.8488	9900.07	7871.25
Gaussian noise ($\sigma_{\text{eff}} = 0.6$)	FG-MoE (full-trained)	**0.8526**	**9774.56**	**7691.53**

Table 7: Master Table 2: Robustness matrix on Wind (CN) under Gaussian perturbations and structured SCADA faults.

Fault Category	**Severity**	**Model**	$R^2 \uparrow$	**RMSE (kW)** $\downarrow$	**Relative Drop (%)**	**Status**
Clean Reference	$\sigma_{\text{eff}} = 0.00$	Ultra-LSNT	0.8728	9081.4	—	✓
	$\sigma_{\text{eff}} = 0.00$	iTransformer	**0.9012**	8004.0	—	✓
	$\sigma_{\text{eff}} = 0.00$	DLinear	0.9400	6236.4	—	✓
Gaussian Noise	$\sigma_{\text{eff}} = 0.15$	Ultra-LSNT	0.8687	9227.3	0.47	✓
	$\sigma_{\text{eff}} = 0.15$	iTransformer	**0.8993**	8053.6	0.21	✓
	$\sigma_{\text{eff}} = 0.15$	DLinear	0.7814	10089.5	16.9	✓
	$\sigma_{\text{eff}} = 0.30$	Ultra-LSNT	0.8552	9759.1	2.01	✓
	$\sigma_{\text{eff}} = 0.30$	iTransformer	**0.8937**	8454.4	0.83	✓
	$\sigma_{\text{eff}} = 0.30$	DLinear	0.2952	17974.2	68.6	×
	$\sigma_{\text{eff}} = 0.60$	Ultra-LSNT	**0.7263**	11853.2	16.8	✓
	$\sigma_{\text{eff}} = 0.60$	iTransformer	**0.8717**	9494.3	3.27	✓
	$\sigma_{\text{eff}} = 0.60$	DLinear	**−1.579**	27109.5	268	××
Structured Faults	Flatline (0.6)	Ultra-LSNT	0.8665	9302.9	0.72	✓
	Flatline (0.6)	iTransformer	0.8942	8278.8	0.77	✓
	Flatline (0.6)	DLinear	**0.9199**	7203.7	1.92	✓
	Spike (0.6)	Ultra-LSNT	**0.7829**	11860.9	10.3	✓
	Spike (0.6)	iTransformer	**0.8609**	9494.3	4.46	✓
	Spike (0.6)	DLinear	0.2058	22686.9	78.1	×
	Dropout (0.6)	Ultra-LSNT	0.8667	9296.5	0.69	✓
	Dropout (0.6)	iTransformer	0.8947	8269.1	0.72	✓
	Dropout (0.6)	DLinear	**0.9245**	7068.2	1.66	✓

Table 8: Physical-violation diagnostics for Ultra-LSNT and FG-MoE on Wind (CN); percentages are computed on inverse-scaled predictions.

Setting	Model	Negative (%)	Over-rated (%)	Cut-region nonzero (%)
Clean	Ultra-LSNT	0.0000	0.1164	0.9387
Clean	FG-MoE (full-trained)	0.0000	**0.0000**	0.9387
$\sigma_{\text{eff}} = 0.6$	Ultra-LSNT	0.0000	0.0653	**1.6896**
$\sigma_{\text{eff}} = 0.6$	FG-MoE (full-trained)	0.0000	**0.0023**	1.7558

Table 9: Gaussian-noise robustness sweep on Wind (CN) with configured ratio σ_{cfg} and effective ratio $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$ ($\alpha = 1.5$).

σ_{cfg} (σ_{eff})	Ultra-LSNT R^2	iTransformer R^2	TimeMixer R^2	PatchTST R^2	DLinear R^2	LightGBM R^2	Persistence R^2
0.00 (0.00)	0.8728	0.9012	0.8886	0.8666	**0.9400**	0.9069	0.9175
0.10 (0.15)	0.8687	0.8993	0.8847	0.8659	0.7814	0.8860	0.9140
0.20 (0.30)	0.8552	0.8937	0.8738	0.8632	0.2952	0.8222	0.9034
0.30 (0.45)	0.7866	0.8844	0.8566	0.8582	-0.4659	0.7305	0.8879
0.40 (0.60)	0.7263	0.8717	0.8337	0.8508	-1.5790	0.6156	0.8612

Table 10: Controlled ablation for iTransformer with and without noise-augmented training under matched inference-time corruption.

Training	Inference σ_{eff}	R^2	RMSE (kW)	vs Clean ΔR^2	Relative Drop (%)
Clean (baseline)	0.00	0.9012	8004.0	—	—
Clean (baseline)	0.30	0.8937	8454.4	−0.0075	0.83
Clean (baseline)	0.60	0.8717	9494.3	−0.0295	3.27
Noise-augmented	0.00	0.8985	8142.3	−0.0027	0.30
Noise-augmented	0.30	0.8912	8587.1	−0.0073	0.81
Noise-augmented	0.60	0.8695	9623.8	−0.0290	3.23

Table 11: Compact efficiency-robustness summary on Wind (CN) using clean and noisy ($\sigma_{\text{eff}} = 0.30$) settings.

Model	Size (MiB)	Latency (ms)	Clean R^2	Noisy R^2	R^2 drop (%)
DLinear	0.80	0.213	0.9400	0.2952	68.6
LightGBM	27.18	50.215	0.9069	0.8222	9.3
Ultra-LSNT	131.62	3.714	0.8728	0.8552	2.0

Table 12: Robustness under structured SCADA faults on Wind (CN) at severity 0.6 (R^2 after inverse scaling).

Fault	Ultra-LSNT R^2	iTransformer R^2	DLinear R^2
Flatline	0.8665	0.8942	**0.9199**
Spike	0.7829	**0.8609**	0.2058
Dropout	0.8667	0.8947	**0.9245**

Table 13: Robustness under missingness and time-axis imputation on Wind (CN) (3 seeds, mean±std).

Imputation	p	Ultra-LSNT R^2	Ultra-LSNT RMSE (kW)	DLinear R^2	DLinear RMSE (kW)
`ffill`	0.1	0.8718 ± 0.0001	9115.0 ± 3.6	0.9326 ± 0.0001	6610.9 ± 4.4
`ffill`	0.3	0.8692 ± 0.0002	9205.8 ± 8.4	0.9242 ± 0.0005	7010.0 ± 23.9
`linear`	0.1	0.8722 ± 0.0001	9102.4 ± 2.9	0.9325 ± 0.0001	6613.6 ± 6.0
`linear`	0.3	0.8705 ± 0.0002	9162.9 ± 6.9	0.9239 ± 0.0006	7021.1 ± 27.9

Table 14: Cross-domain generalization results across Wind (CN), Wind (US), Air Quality, and GEFCom Load (R^2, higher is better).

Dataset	Ultra-LSNT R^2	DLinear R^2	TimeMixer R^2	Ultra skip rate (%)
Wind (CN)	0.8728	0.9400	0.8886	0.0
Wind (US)	0.7878	0.8796	0.8023	11.4
Air Quality	0.4861	0.3400	0.3869	0.0
GEFCom Load	0.7554	0.7363	0.7271	0.0

Table 15: Dataset-native error metrics for Ultra-LSNT after inverse scaling; units follow each dataset.

Dataset	RMSE	MAE	Unit	Skip rate (%)
Wind (CN)	9081	7171	kW	0.0
Wind (US)	0.4556	0.2891	MW	11.4
Air Quality	32.21	19.87	AQI	0.0
GEFCom Load	2837	2049	MW	0.0

Table 16: Ramp, tail-error, smoothness, and routing-stability diagnostics on Wind (CN).

Model	$P95(\lvert e\rvert)$ (kW)	$\max(\lvert e\rvert)$ (kW)	Ramp F1	TV (kW)	TV/TV_{true}	Routing var	(max-share, entropy)
Ultra-LSNT	11,940.17	34,544.14	0.3544	8,222,569.29	0.2525	0.04413	(0.5394, 0.7511)
DLinear	2,484.47	21,479.02	0.5939	29,013,519.33	0.8909	0.00000	(1.0000, 0.0000)
Static ens. (0.5U+0.5D)	6,151.70	28,011.58	0.5000	15,059,403.24	0.4624	0.00000	(0.5000, 1.0000)

Table 17: Post-hoc Gaussian calibration baseline on Wind (CN) with nominal 80% and 90% intervals.

Model	σ (kW)	NLL	CRPS	PICP@80 / PINAW@80	PICP@90 / PINAW@90
Ultra-LSNT	5,044.08	10.1314	3,353.4667	0.7253 / 0.10995	0.8242 / 0.14111
DLinear	1,329.74	8.6118	670.2737	0.8726 / 0.02898	0.9269 / 0.03720

Table 18: Master Table 3: Cross-ISA hardware profiling for edge deployment on x86 and ARMv8, with RTX 4090 throughput as reference.

Platform	**Model**	**Params (M)**	**Size (MiB)**	**Batch**	**Latency (ms)**	**Throughput (Hz)**	**Energy*** **(mJ/sample)**	**Edge-Ready**
GPU (RTX 4090)	Ultra-LSNT	34.46	131.62	128	18.84 (fwd)	6,793	1.47	Reference
	Ultra-LSNT-Lite	1.34	5.10	256	10.42 (fwd)	24,559	0.35	Reference
	Ultra-LSNT-Lite (Opt.)	0.35	1.33	256	6.53 (fwd)	39,204	0.25	Reference
	DLinear	0.21	0.80	128	0.05 (fwd)	2,560,000	0.01	Reference
CPU x86 (Batch-1)	Ultra-LSNT (Full)	34.46	131.62	1	3.714 ± 0.34	269.3	–	✓
	Ultra-LSNT-Lite	1.34	5.10	1	2.430 ± 0.25	411.5	–	✓
	Ultra-LSNT-Lite (Opt.)	0.35	1.33	1	**1.171** ± 0.20	**854.0**	–	✓✓
	DLinear	0.21	0.80	1	0.265 ± 0.08	3767.6	–	✓✓
	LightGBM	–	27.18	1	50.40 ± 21.1	19.8	–	×
ARMv8 (Kunpeng 920)†	Ultra-LSNT (Full)	34.46	104.5	1	3.275 (P95: 4.35)	305.3	–	✓
	Ultra-LSNT-Lite	0.35	15.7	1	**2.348** (P95: 3.14)	425.9	–	✓✓

*Analytical proxy using 10W TDP reference (ARM-class industrial controller); HIL measurement required for actual validation.

†Cross-ISA validation (aarch64) confirms computational graph portability; Kunpeng 920 used as server-class ARM reference, not edge surrogate.

Table 19: Single-thread CPU batch-1 microbenchmark for online inference latency.

Model	Latency (ms)	Throughput (Hz)
Ultra-LSNT (full)	3.714 ± 0.341	269.3
Ultra-LSNT-Lite (orig.)	2.430 ± 0.245	411.5
Ultra-LSNT-Lite (opt.)	1.171 ± 0.201	854.0
DLinear	0.2654 ± 0.0843	3767.6
LightGBM	50.3970 ± 21.1347	19.8

Table 20: ARMv8 (aarch64) single-core batch-1 microbenchmark on Huawei Kunpeng 920 for cross-ISA latency screening.

Model	Avg latency (ms)	P95 latency (ms)	Artifact size (MB)
Ultra-LSNT (Full)	3.275	4.349	1.021
Ultra-LSNT-Lite	2.348	3.137	0.154

Table 21: Jump Gate threshold sweep on Wind (CN) with R^2, RMSE, skip rate, and CPU batch-1 latency.

τ	R^2	RMSE (kW)	Skip rate (%)	CPU latency (ms)
0.3 (Edge-Frugal)	0.8661	9314.40	33.38	3.621
0.5 (Reliability, default)	0.8728	9081.66	0.00	3.628
0.7	0.8728	9081.66	0.00	3.766
0.9	0.8728	9081.66	0.00	3.665

Table 22: Master Table 4: Economic dispatch value transfer in copper-plate rolling evaluation and RTS-24 DC-OPF screening.

Layer	**Model**	**Operating Cost**	**Curtailment**	**Fallback Frequency**	**vs DLinear Improvement**	**Latency (ms)**	**Params (M)**
L1: Copper-Plate	iTransformer	32.16M CNY	15.21M kWh	16.7%	+72.7%	54.2	84.5
	TimeMixer	34.90M CNY	19.60M kWh	21.7%	+64.8%	12.3	2.6
	Ultra-LSNT	35.97M CNY	22.09M kWh	24.2%	**+60.4%**	**<4.0**	**0.35**
	DLinear (baseline)	54.47M CNY	55.71M kWh	62.5%	—	0.3	0.2
L2: IEEE RTS-24	iTransformer	145.43M*	**54.24** MWh	100%†	+12.2%	54.2	84.5
	TimeMixer	147.91M*	**54.24** MWh	100%†	+12.2%	12.3	2.6
	Ultra-LSNT	145.59M*	54.38 MWh	100%†	**+12.0%**	**<4.0**	**0.35**
	DLinear (baseline)	137.08M*	61.80 MWh	100%†	—	0.3	0.2

*RT cost in RTS-24 units (not directly comparable to CNY); †OPF success ratio.

Layer 1 demonstrates workflow feasibility expansion; Layer 2 demonstrates congestion-screening value under network constraints.

Table 23: Cross-regime comparison between edge-feasible Ultra-LSNT-Lite and cloud-scale Chronos-Mini (TSFM) to illustrate deployment feasibility boundaries.

Model	Parameters (M)	Batch-1 Latency (ms)	Memory footprint (MiB)	R^2 (Wind CN)
Chronos-Mini (TSFM)	350	1200	1300	0.95
Ultra-LSNT-Lite	0.35	1.17	1.33	0.87
Edge-tier limit	<10	<10	<1024	–

Table B.24: Computational hardware used for training and inference profiling.

Hardware component	Specification
GPU	NVIDIA GeForce RTX 4090 (or equivalent)
CPU	Intel Core i9-13900K (or equivalent)
Memory	64 GB DDR5 RAM
Storage	2 TB NVMe SSD
OS	Ubuntu 22.04 LTS / Windows 11 (WSL2)

Table B.25: Ultra-LSNT and Ultra-LSNT-Lite configuration summary used for reproducibility.

Hyperparameter	Ultra-LSNT (main)	Ultra-LSNT-Lite (Opt.)
Lookback L / horizon H	96 / 24	96 / 24
Model dimension d	128	96
# MoE blocks / layers	3	1
# experts N	4	2
Top-K routing K	2	1
Router auxiliary loss λ	0.01	0.01
Jump Gate threshold τ	0.5	0.5
Dropout	0.1	0.1
Optimizer / LR	AdamW / 1×10^{-3}	AdamW / 1×10^{-3}
Batch size	128	128
Inference optimization	Eager FP32 (no compile/AMP)	`torch.compile`+AMP (FP16)

Table D.26: Full structured SCADA-fault robustness results on Wind (CN) for Ultra-LSNT, iTransformer, and DLinear.

Fault type	Severity	Model	R^2	RMSE (kW)	MAE (kW)	ΔR^2_{abs}	ΔR^2_{rel} (%)
clean	0.0	Ultra-LSNT	0.8727	9081.4	7168.7	0.0000	0.00
clean	0.0	iTransformer	0.9012	8004.0	6333.3	0.0000	0.00
clean	0.0	DLinear	0.9379	6342.9	4203.5	0.0000	0.00
flatline	0.2	Ultra-LSNT	0.8721	9103.3	7181.3	0.0006	0.07
flatline	0.2	iTransformer	0.9005	8029.5	6351.9	0.0006	0.07
flatline	0.2	DLinear	0.9369	6394.6	4230.8	0.0010	0.11
flatline	0.6	Ultra-LSNT	0.8665	9302.9	7345.3	0.0063	0.72
flatline	0.6	iTransformer	0.8942	8278.8	6541.4	0.0069	0.77
flatline	0.6	DLinear	0.9199	7203.7	4639.0	0.0180	1.92
spike	0.2	Ultra-LSNT	0.8509	9831.3	7737.6	0.0219	2.51
spike	0.2	iTransformer	0.8897	8454.4	6594.6	0.0114	1.27
spike	0.2	DLinear	0.7766	12031.7	5335.3	0.1613	17.20
spike	0.6	Ultra-LSNT	0.7829	11860.9	9324.8	0.0898	10.29
spike	0.6	iTransformer	0.8609	9494.3	7228.6	0.0402	4.46
spike	0.6	DLinear	0.2058	22686.9	7864.8	0.7321	78.05
dropout	0.2	Ultra-LSNT	0.8721	9105.5	7190.2	0.0007	0.08
dropout	0.2	iTransformer	0.9004	8034.7	6363.1	0.0008	0.08
dropout	0.2	DLinear	0.9358	6450.5	4275.6	0.0021	0.23
dropout	0.6	Ultra-LSNT	0.8667	9294.1	7334.4	0.0060	0.69
dropout	0.6	iTransformer	0.8947	8259.4	6537.0	0.0064	0.71
dropout	0.6	DLinear	0.9245	6993.4	4557.9	0.0134	1.43

Table D.27: Kernel-size ablation on Wind (CN).

Kernel size	R^2
7	0.8591
3	0.8505
15	0.8214

Table D.28: Router regularization coefficient (λ) ablation on Wind (CN, clean setting).

λ	R^2
0.00	0.8767
0.01	0.8715
0.05	0.8832
0.10	0.8731

Table D.29: Expert specialization summary using the highest-correlated physical feature.

Expert	Feature	Correlation
Expert 1	wind_speed	+0.42
Expert 2	turbulence	+0.38
Expert 3	temperature	+0.31
Expert 4	humidity	+0.25

Table D.30: Diebold–Mariano test results on Wind (CN) using squared-error loss; DM< 0 favors Ultra-LSNT.

Comparison	DM statistic	p-value
Ultra-LSNT vs PatchTST	-5.90	1.82×10^{-9}
Ultra-LSNT vs TimeMixer	3.45	2.83×10^{-4}
Ultra-LSNT vs iTransformer	5.12	3.07×10^{-7}